

Contents		
1	Formulas	2
1.1	bitwise	2
1.2	chapter	2
1.2.1	Trigonometry	2
1.3	Combinatorics	2
1.4	CombiOhin	3
1.5	FahimVy	3
1.5.1	SUM	3
1.5.2	Number Theory	3
1.5.3	Totient	3
1.6	Fibonacci	3
1.7	NT	3
2	Build System	4
2.1	StressTest	4
2.1.1	gen	4
2.1.2	script	4
2.2	OJ	4
2.3	sublime	4
3	Combinatorics	4
3.1	catalan	4
3.2	gen_all_k_combs	4
3.3	IntPerm	4
3.4	Modular Inverse	4
3.5	nCr mod	4
3.6	next_lexicographical_k_comb	4
3.7	pruferCode	4
3.8	Stirling2nd	4
4	Data Structure	5
4.1	2D BIT	5
4.2	2dsegmentTree	5
4.3	BasicSegtreeOhin	5
4.4	BIT	5
4.5	Cartesian Tree	5
4.6	DSUParity	6
4.7	DynamicConnectivity	6
4.8	lazyRangeAdd	6
4.9	LineContainer	6
4.10	moalgo	7
4.11	Persistent Segtree	7
4.12	Segment Tree Merge Sort Tree	7
4.13	SegmentTreeTamzid	8
4.14	SparseWithIndex	8
4.15	Treap	8
4.16	TrieBit	8
5	Dynamic Programming	9
5.1	CHT	9
5.2	DivideNConquer	9
5.3	LiChaoTree	9
5.4	SOSDP	9
6	Geometry	10
6.1	3D	10
6.1.1	3D Convex Hull	10
6.2	Circle	10
6.2.1	Circle Intersection	10
6.2.2	Circle Polygon Intersection	10
6.2.3	Circle Tangents	10
6.3	Polygon	10
6.3.1	Hull Diameter	10
6.3.2	Line Hull Intersection	10
6.3.3	Polygon Center	11
6.4	Closest_Pair	11
6.5	Half Plane	11
6.6	inside_poly_fast	11
6.7	Manhattan MST	11
6.8	MEC	12
6.9	Minkowski	12
6.10	Point 2D	12
6.11	Point 3D	13
6.12	Polygon Union	13
6.13	Re-order Polygon	13
6.14	Rectangle Union	14
6.15	Rotating_Calipers	14
6.16	Ternary Search	14
7	Graph	14
7.1	Articulation_Bridge	14
7.2	Articulation_Point	14
7.3	bellmanford	14
7.4	BFS for All	15
7.5	BiconnectedComponents	15
7.6	Binary Lifting - Lowest Common Ancestor (LCA)	15
7.7	CentroidDecomposition	15
7.8	DFS Plus for tree	15
7.9	Dinic	16
7.10	DPOnTree	16
7.11	DSU	16
7.12	DsuOnTree	16
7.13	eulerpathdirected	16
7.14	eulerpathundirected	17
7.15	FloydWarshall	17
7.16	gridBFS+DFS	17
7.17	HLD2	17
7.18	MST Prim's	17
7.19	SCC2	17
7.20	Subtree queries - Number of nodes with color k in the subtree	18
7.21	TopoSort	18
7.22	TreeDiameter	18
7.23	UnionFindRollback	18
8	Math	18
8.1	Binary Operations	18
8.2	FFT	18
8.3	Gaussian Elimination	19
8.4	Matrix	19
8.5	Mobius Precomputation	19
8.6	ntt	19
8.7	PolyRoots	20
8.8	XorBasis	20

9 Number Theory	20	10.4 PolyHash	22
9.1 crt	20	10.5 Prefix Automaton	22
9.2 extendedEuclidean	20	10.6 SuffixArray	22
9.3 FastEratosthenes	20	10.7 SuffixAuto	23
9.4 FloorSum	20	10.8 trieString	24
9.5 millerRabin	21		
9.6 NumberTheory	21	11 zMiscellaneous	24
9.7 PrimitiveRoot	21	11.1 bitset_operations	24
9.8 recursive divisor generate	21	11.2 builtinOperations	24
9.9 Sieve	21	11.3 CoordinateCompression	24
9.10 SquareFreeCount	21	11.4 CUSTOM HASH	25
9.11 Totient Values	21	11.5 Custom_Comparator_for_pq	25
10 String	21	11.6 fast_subset_sum	25
10.1 KMP and Z Function	21	11.7 int128	25
10.2 Manacher	22	11.8 pbds	25
10.3 MinRotation	22	11.9 submaskiterate	25

1 Formulas**1.1 bitwise**

- $a + b = a \oplus b + 2 \cdot (a \& b)$
- $a + b = a | b + a \& b$
- $a \oplus b = a | b - a \& b$
- $n \bmod 2^i = n \& (2^i - 1)$
- $1 \oplus 2 \oplus 3 \oplus \dots \oplus (4k - 1) = 0$ for any $k \geq 0$

1.2 chapter**1.2.1 Trigonometry**

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}$, $\phi = \text{atan2}(b, a)$.

1.3 Combinatorics

$$\sum_{k=0}^n \binom{n-k}{k} = \text{Fib}_{n+1}$$

$$\binom{n}{k} = \binom{n}{n-k}$$

$$\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$$

$$k \binom{n}{k} = n \binom{n-1}{k-1}$$

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$$

$$\sum_{i=0}^n \binom{n}{i} = 2^n$$

$$\sum_{i=0}^{\lfloor n/2 \rfloor} \binom{n}{2i} = 2^{n-1}$$

$$\sum_{i=0}^{\lfloor (n-1)/2 \rfloor} \binom{n}{2i+1} = 2^{n-1}$$

$$\sum_{i=0}^k (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$$

$$\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$$

$$\sum_{i=1}^n i \binom{n}{i} = n \cdot 2^{n-1}$$

$$\sum_{i=1}^n i^2 \binom{n}{i} = (n + n^2) \cdot 2^{n-2}$$

$$\text{Vandermonde's Identity: } \sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$$

$$\text{Hockey-Stick Identity: } \sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$$

$$\sum_{i=0}^k \binom{k}{i}^2 = \binom{2k}{k}$$

$$\sum_{k=0}^n \binom{n}{k} \binom{n}{n-k} = \binom{2n}{n}$$

$$\sum_{k=q}^n \binom{n}{k} \binom{k}{q} = 2^{n-q} \binom{n}{q}$$

$$\sum_{i=0}^n k^i \binom{n}{i} = (k+1)^n$$

$$\sum_{i=0}^n \binom{2n}{i} = 2^{2n-1} + \frac{1}{2} \binom{2n}{n}$$

$$\sum_{i=1}^n \binom{n}{i} \binom{n-1}{i-1} = \binom{2n-1}{n-1}$$

$$\sum_{i=0}^n \binom{2n}{i}^2 = \frac{1}{2} \left(\binom{4n}{2n} + \binom{2n}{n}^2 \right)$$

$$\text{Catalan Number: } C_n = \frac{1}{n+1} \binom{2n}{n}$$

$$\text{Derangement: } d(n) = (n-1) \cdot (d(n-1) + d(n-2)), \text{ with } d(0) = 1, d(1) = 0$$

$$\text{Stirling Numbers of the First Kind: } s(n, k) = s(n-1, k-1) + (n-1) \cdot s(n-1, k)$$

$$\text{Stirling Numbers of the Second Kind: } S(n, k) = S(n-1, k-1) + k \cdot S(n-1, k)$$

• Bell Numbers: $B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k$ • $\sum_{i=0}^n i \cdot i! = (n+1)! - 1$ • $\sum_{i=1}^n i a^i = \frac{a(na^{n+1} - (n+1)a^n + 1)}{(a-1)^2}$ • Fibonacci: $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$ • $F_n = \sum_{k=0}^{\lfloor (n-1)/2 \rfloor} \binom{n-k-1}{k}$ • $F_n = \frac{1}{\sqrt{5}} \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right)$ • $\sum_{i=1}^n F_i = F_{n+2} - 1$ • $\sum_{i=0}^{n-1} F_{2i+1} = F_{2n}$ • $\sum_{i=1}^n F_{2i} = F_{2n+1} - 1$ • Pick's Theorem: For a simple polygon with vertices on lattice points, $A = I + \frac{B}{2} - 1$ where A is area, I is number of interior lattice points, and B is number of boundary lattice points. 1.4 CombiOhin Derrangement Derangements recurrence relation $D_n = (n-1)(D_{n-1} + D_{n-2})$ Derangements explicit formula $D_n = n! - \binom{n}{1}(n-1)! + \binom{n}{2}(n-2)! - \dots + (-1)^n \binom{n}{n} 0!$ n length derangement with exactly k excedents $dp[n][k] = (n-1) * dp[n-2][k-1] + (n-k) * dp[n-1][k-1] + k * dp[n-1][k]$ Eulerian numbers recurrence: n-length permutation with exactly k-excedents $A(n, k) = (k+1)A(n-1, k) + (n-k)A(n-1, k-1)$	Stirling Number + Stars and Bars n-distinct objects into k non-empty sets. $S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$ Number of permutations of n elements with exactly k cycles $c(n, k) = (n-1)c(n-1, k) + c(n-1, k-1)$ n identical objects and k non-empty distinct bins ($n-1$ stars and $k-1$ bars) $\text{Number of ways} = \binom{n-1}{k-1}$ n identical objects and k distinct bins (n stars and $k-1$ bars) $\text{Number of ways} = \binom{n+k-1}{k-1} = \binom{n+k-1}{n}$ Convolution Going over all partitions summing to s: $f[t][s] = \sum_{i=0}^s f[t-1][s-i] \cdot dp[i]$ Convex Hull Trick / Li Chao Tree CHT/Li Chao Tree eqn, mixed $\rightarrow$ slope, only $j \rightarrow$ intercept, constant $\rightarrow$ only i $\text{Cost} = \underbrace{(cost_j \cdot x_i)}_{\text{Mixed}} + \underbrace{(dp_j - cost_j \cdot x_j)}_{\text{Only } j} + \underbrace{toll_i}_{\text{Only } i}$ Catalan Numbers / Dyck Path (1, 1) and (1, -1) movement but cant go below $y = 0$. (0, 0) $\rightarrow$ (2n, 0) if bad path, bijection: go from (0, 0) $\rightarrow$ (2n, -2), reflect at first -1 $C_n = \frac{1}{n+1} \binom{2n}{n}$ $C_0 = 1, \quad C_{n+1} = \sum_{i=0}^n C_i C_{n-i}$ $C_{n+1} = \frac{2(2n+1)}{n+2} C_n$ 1.5 FahimVy 1.5.1 SUM $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$	1.5.2 Number Theory - Primitive root exists iff $n = 1, 2, 4, p^k, 2 \times p^k$, where p is an odd prime. 1.5.3 Totient - Sum of gcd, $\sum_{i=1}^n gcd(i, n) = \sum_{d n} d \phi\left(\frac{n}{d}\right)$ - Sum of lcm, $\sum_{i=1}^n n lcm(i, n) = \frac{n}{2} (\sum_{d n} (d \phi(d)) + 1)$ 1.6 Fibonacci Let A, B and n be integer numbers. $k = A - B$ $F_A F_B = F_{k+1} F_A^2 + F_k F_A F_{A-1}$ $\sum_{i=0}^n F_i^2 = F_{n+1} F_n$ $\sum_{i=0}^n F_i F_{i+1} = F_{n+1}^2 - (-1)^n$ $\sum_{i=0}^n F_i F_{i-1} = \sum_{i=0}^{n-1} F_i F_{i+1}$ $gcd(F_m, F_n) = F_{gcd(m, n)}$ $\sum_{0 \leq k \leq n} \binom{n-k}{k} = F_{n+1}$ $gcd(F_n, F_{n+1}) = gcd(F_n, F_{n+2}) = gcd(F_{n+1}, F_{n+2}) = 1$ 1.7 NT $\sum_{k=1}^n \frac{1}{gcd(k, n)} = \sum_{d n} \frac{1}{d} \varphi\left(\frac{n}{d}\right) = \frac{1}{n} \sum_{d n} d \varphi(d)$ $\sum_{k=1}^n \frac{k}{gcd(k, n)} = 1 + \frac{1}{2} \sum_{\substack{d n \\ d>1}} d \varphi(d)$ $\sum_{k=1}^n \frac{n}{gcd(k, n)} = 2 \sum_{k=1}^n gcd(k, n) - 1, \quad \text{for } n > 1$ $\sum_{i=1}^n \sum_{j=1}^n [gcd(i, j) = 1] = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2 = 2 \sum_{i=1}^n \varphi(i) - 1$ $\sum_{i=1}^n \sum_{j=1}^n gcd(i, j) = \sum_{d=1}^n \varphi(d) \left\lfloor \frac{n}{d} \right\rfloor^2$ $\sum_{i=1}^n \sum_{j=1}^n i \cdot j [gcd(i, j) = 1] = \sum_{d=1}^n \mu(d) \cdot d^2 \left(\frac{\lfloor n/d \rfloor (\lfloor n/d \rfloor + 1)}{2} \right)^2$ $\sum_{i=1}^n \sum_{j=1}^n lcm(i, j) = \sum_{d=1}^n d \sum_{k=1}^{\lfloor n/d \rfloor} \mu(k) \cdot k^2 \left(\frac{\lfloor \frac{n}{d \cdot k} \rfloor (\lfloor \frac{n}{d \cdot k} \rfloor + 1)}{2} \right)^2$
---	--	---

$$\sum_{d|n} \mu(d) = [n = 1]$$

$$\sum_{d|n} \varphi(d) = n$$

$$\sum_{d|n} \mu(d) \frac{n}{d} = \varphi(n)$$

$$g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right)$$

$$g(n) = \sum_{k=1}^{\lfloor M/n \rfloor} f(k \cdot n) \iff f(n) = \sum_{k=1}^{\lfloor M/n \rfloor} \mu(k) g(k \cdot n)$$

$$\sum_{i=1}^n \gcd(i, n) = \sum_{d|n} d \cdot \varphi\left(\frac{n}{d}\right)$$

$$\sum_{i=1}^n \text{lcm}(i, n) = \frac{n}{2} \left(\sum_{d|n} d \cdot \varphi(d) + 1 \right)$$

$$\sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = 1] = \sum_{d=1}^{\min(n, m)} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

$$\sum_{i=1}^n \sum_{j=1}^m \gcd(i, j) = \sum_{d=1}^{\min(n, m)} \varphi(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

$$v_p(n!) = \sum_{k=1}^{\infty} \left\lfloor \frac{n}{p^k} \right\rfloor$$

$$\binom{n}{k} \equiv \prod_{i=0}^{r-1} \binom{n_i}{k_i} \pmod{p}$$

2 Build System

2.1 StressTest

2.1.1 gen

```
int rnd(int a, int b){
    return a + rand() % (b - a + 1);}
int main(int argc, char *argv[]){
    int seed = atoi(argv[1]);
    srand(seed);
    int n = rnd(1, 500);
    cout << n << endl; }
```

2.1.2 script

```
# script.sh
set -e
g++ code.cpp -o code
g++ gen.cpp -o gen
g++ brute.cpp -o brute
for((i = 1; ; ++i)); do
    ./gen $i > input file
    ./code < input file > myAnswer
    ./brute < input file > correctAnswer
    diff -Z myAnswer correctAnswer > /dev/null || break
    echo "Passed test: " $i
done
echo "WA on the following test:"
cat input file
echo "Your answer is:"
cat myAnswer
echo "Correct answer is:"
cat correctAnswer
```

2.2 OJ

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    #ifndef ONLINE_JUDGE
        freopen("input.txt", "r", stdin);
        freopen("output.txt", "w", stdout);
    #endif
    int n; cin >> n;
    for(int i=0; i<n; i++){
        cout << i+1 << endl;
    }
    cout << "hello" << endl; }
```

2.3 sublime

```
{"cmd" : ["g++ -std=c++14 $file_name -o $file_base_name
    && timeout 4s
    ./ $file_base_name<input.txt>output.txt"],
"selector" : "source.c",
"shell" : true,
"working_dir" : "$file_path" }
```

3 Combinatorics

3.1 catalan

```
void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=n; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1]) % MOD;
        }
        if (catalan[i] >= MOD) {
            catalan[i] -= MOD;
        }
    }
}
```

3.2 gen_all_k_combs

```
vector<int> ans;
void gen(int n, int k, int idx, bool rev) {
    if (k > n || k < 0)
        return;
    if (!n) {
        for(int i = 0; i < idx; ++i) {
            if (ans[i])
                cout << i + 1;
            cout << "\n";
        }
        return;
    }
```

```
ans[idx]=rev; gen(n-1, k- rev, idx+1, false);
ans[idx] = !rev; gen(n-1, k-!rev, idx+1, true);}
void all_combinations(int n, int k) {
    ans.resize(n);
    gen(n, k, 0, false);}
```

3.3 IntPerm

```
//Description: Permutation -> integer conversion. (Not
    ~ order preserving.)
//Integer -> permutation can use a lookup table.
int permToInt(vi &v) {
    int use = 0, i = 0, r = 0;
    for (int x : v)
        r = r * ++i + __builtin_popcount(use & -(1 <<
            ~ x)),
        use |= 1 << x; // (note: minus, not ~!)
    return r; }
```

3.4 Modular Inverse

```
ll inv(ll a, ll m){
    if(a<=1) return a;
    return m - ((ll)(m/a)*inv(m%a,m)%m);}
ll A_by_B_mod(ll a, ll b){
    return (a*(binpow(b,M-2,M)))%M; }
```

3.5 nCr mod

```
// call FACTMOD() in int main()
vll factm(N,1ll);
vll invf(N,1ll);
ll binpow(ll a, ll n, ll m){
    if(n==0) return 1ll;
    ll x=binpow(a,n/2,m);
    if(n&1ll) return (a*((x*x)%m))%m;
    else return (x*x)%m;
}
void FACTMOD(ll n, ll m){
    loop(i,2,n)
        factm[i]=(factm[i-1]*i)%m;
    invf[n-1]=binpow(factm[n-1],m-2,m);
    for(ll i=n-1;i>=1;i--)
        invf[i-1]=(invf[i]*i)%m;
}
ll nCr(ll n, ll r, ll m){
    if(r<0 || r>n) return 0ll;
    return factm[n]*invf[r]%m*invf[n-r]%m; }
```

3.6 next_lexicographical_k_comb

```
bool next_combination(vector<int>& a, int n) {
    int k = (int)a.size();
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i + 1) {
            a[i]++;
            for (int j = i + 1; j < k; j++)
                a[j] = a[j - 1] + 1;
            return true;
        }
    }
    return false; }
```

3.7 pruferCode

```
vector<int> g[N];
int parent[N], degree[N];
// 0-indexed
void dfs(int v) {
    for (int i = 0; i < g[v].size(); i++) {
        int to = g[v][i];
        if (to != parent[v]) {
            parent[to] = v;
            dfs(to);
        }
    }
}
vector<int> pruferCode(int n) {
    parent[n - 1] = -1;
    dfs(n - 1);
    int ptr = -1;
    for (int i = 0; i < n; i++) {
        degree[i] = (int)g[i].size();
        if (degree[i] == 1 && ptr == -1)
            ptr = i;
    }
    vector<int> result;
    int leaf = ptr;
    for (int iter = 0; iter < n - 2; iter++) {
        int next = parent[leaf];
        result.push_back(next);
        --degree[next];
        if (degree[next] == 1 && next < ptr)
            leaf = next;
        else {
            ++ptr;
            while (ptr < n && degree[ptr] != 1)
                ++ptr;
            leaf = ptr;
        }
    }
    return result;
}
```

3.8 Stirling2nd

```
ll get_sn2(int n, int k) {
    ll sn2 = 0;
    for (int i = 0; i <= k; ++i) {
        ll now = nCr(k, i) * powmod(k - i, n, mod) %
            mod;
        if (i & 1) now = now * (mod - 1) % mod;
        sn2 = (sn2 + now) % mod;
    }
    sn2 = sn2 * ifact[k] % mod;
    return sn2;
}
```

4 Data Structure

4.1 2D BIT

```
// Grid Indexing: 0-based indexing
// fenwick tree 2d<ll> bit2d(n+1,m+1);
// Supported Operations:
// - add(x, y, val) -> A[x][y] += val
// - sum(x, y) -> sum of submatrix
// (0,0) to (x,y)
// - get_sum(x1, y1, x2, y2) -> sum of submatrix
// (x1,y1) to (x2,y2), inclusive
template <class T>
struct fenwick tree 2d {
    vector<vector<T>>> x;
    fenwick tree 2d(int n, int m) : x(n, vector<T>(m)) {}
    void add(int k1, int k2, T a) { // x[k1][k2] += a
        for (; k1 < x.size(); k1 |= k1 + 1)
            for (int k = k2; k < x[k1].size(); k |= k + 1)
                x[k1][k] += a;
        T sum(int k1, int k2) { // sum from (0,0) to
            (k1,k2)
            T s = 0;
            for (; k1 >= 0; k1 = (k1 & (k1 + 1)) - 1)
                for (int k = k2; k >= 0; k = (k & (k + 1)) - 1)
                    s += x[k1][k];
            return s;
        }
        T get_sum(int x1, int y1, int x2, int y2) { // sum
            in rectangle (x1,y1) to (x2,y2)
            return sum(x2, y2)
                - (x1 > 0 ? sum(x1 - 1, y2) : 0)
                - (y1 > 0 ? sum(x2, y1 - 1) : 0)
                + (x1 > 0 && y1 > 0 ? sum(x1 - 1, y1 - 1) : 0);
        }
    }
};
```

```
for (; k1 >= 0; k1 = (k1 & (k1 + 1)) - 1)
    for (int k = k2; k >= 0; k = (k & (k + 1)) - 1)
        s += x[k1][k];
return s;
}
T get_sum(int x1, int y1, int x2, int y2) { // sum
    in rectangle (x1,y1) to (x2,y2)
    return sum(x2, y2)
        - (x1 > 0 ? sum(x1 - 1, y2) : 0)
        - (y1 > 0 ? sum(x2, y1 - 1) : 0)
        + (x1 > 0 && y1 > 0 ? sum(x1 - 1, y1 - 1) : 0);
}
```

4.2 2dsegmentTree

```
struct SegTree2D {
    ll siz;
    vector<vector<ll>>> tree;
    void initialize(ll _n) {
        siz = 1;
        while (siz < _n)
            siz *= 2;
        tree.assign(2 * siz, vector<ll>(2 * siz, 0));
    }
    void update_x(ll y_node, ll x_node, ll start, ll end,
        ll x, ll val) {
        if (start == end) {
            tree[y_node][x_node] += val;
            return;
        }
        ll mid = (start + end) / 2;
        if (x <= mid)
            update_x(y_node, 2 * x_node + 1, start, mid,
                x, val);
        else
            update_x(y_node, 2 * x_node + 2, mid + 1, end,
                x, val);
        tree[y_node][x_node] = tree[y_node][2 * x_node +
            1] + tree[y_node][2 * x_node + 2];
    }
    ll get_x(ll y_node, ll x_node, ll start, ll end, ll
        x1, ll x2) {
        if (end < x1 || start > x2)
            return 0;
        if (start >= x1 && end <= x2)
            return tree[y_node][x_node];
        ll mid = (start + end) / 2;
        return get_x(y_node, 2 * x_node + 1, start, mid,
            x1, x2) +
            get_x(y_node, 2 * x_node + 2, mid + 1, end,
                x1, x2);
    }
    void update_y(ll node, ll start, ll end, ll y, ll x,
        ll val) {
        update_x(node, 0, 0, siz - 1, x, val);
        if (start == end)
            return;
        ll mid = (start + end) / 2;
        if (y <= mid)
            update_y(2 * node + 1, start, mid, y, x, val);
        else
            update_y(2 * node + 2, mid + 1, end, y, x,
                val);
    }
    ll get_y(ll node, ll start, ll end, ll y1, ll y2, ll
        x1, ll x2) {
        if (end < y1 || start > y2)
            return 0;
        if (start >= y1 && end <= y2)
            return get_x(node, 0, 0, siz - 1, x1, x2);
        ll mid = (start + end) / 2;
        return get_y(2 * node + 1, start, mid, y1, y2, x1,
            x2) +
            get_y(2 * node + 2, mid + 1, end, y1, y2,
                x1, x2);
    }
    void update(ll y, ll x, ll val) {
        update_y(y, 0, 0, siz - 1, x, val);
    }
};
```

```
update y(0, 0, siz - 1, y, x, val);
}
ll query(ll y1, ll x1, ll y2, ll x2) {
    return get_y(0, 0, siz - 1, y1, y2, x1, x2);
}
```

4.3 BasicSegtreeOhin

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef vector<ll> vll;
struct SegTree {
    ll sz = 0;
    vll res;
    ll dummy;
    void init(int n, ll d) {
        dummy = d;
        sz = 1;
        while (sz < n) sz *= 2;
        res.assign(sz * 2, d);
    }
    ll op(ll a, ll b) {
        return a + b;
    }
    void build(vll &a, int x, int lx, int rx) {
        if (lx == rx) {
            if (lx < (int)a.size())
                res[x] = a[lx];
            else
                res[x] = dummy;
            return;
        }
        ll mid = (lx + rx) / 2;
        build(a, 2 * x + 1, lx, mid);
        build(a, 2 * x + 2, mid + 1, rx);
        res[x] = op(res[2 * x + 1], res[2 * x + 2]);
    }
    void update(int i, ll v, int x, int lx, int rx) {
        if (lx == rx) {
            res[x] = v;
            return;
        }
        ll mid = (lx + rx) / 2;
        if (i <= mid) update(i, v, 2 * x + 1, lx, mid);
        else update(i, v, 2 * x + 2, mid + 1, rx);
        res[x] = op(res[2 * x + 1], res[2 * x + 2]);
    }
    ll query(int l, int r, int x, int lx, int rx) {
        if (r < lx || rx < l)
            return dummy;
        if (lx >= l && rx <= r)
            return res[x];
        ll mid = (lx + rx) / 2;
        return op(query(l, r, 2 * x + 1, lx, mid),
            query(l, r, 2 * x + 2, mid + 1, rx));
    }
    //first value greater than v, or lower_bound of prefix
    // sum v, or kth number
    ll walk(ll v, int x, int lx, int rx) {
        if (res[x] < v) return -1;
        if (lx == rx)
            return lx;
        ll mid = (lx + rx) / 2;
        if (res[2 * x + 1] < v) return walk(v - res[2 * x
            + 1], 2 * x + 2, mid + 1, rx);
        else return walk(v, 2 * x + 1, lx, mid);
    }
};
```

4.4 BIT

```
template <class T> struct BIT {
    // A[] has to be 1-indexed
    // create BIT of n elements : BIT<ll> bit(n);
    // MODE A (point-update + prefix/range-sum):
    // add v to A[idx]: bit.upd(idx, v);
    // sum of A[1..idx]: ll sum1 = bit.query(idx);
    // sum of A[l..r]: ll sumLR = bit.query(l,
        r);
};
```

```
// MODE B (range-update + point-query):
// add v to every A[i] for i in [l..r]:
//   bit.upd(l, r, v);
//   current A[idx]: ll Ai = bit.query(idx);
// Note: Do NOT mix MODE A and MODE B on the same
// instance-
// use separate BITS if you need both.
int n;
vector<T> t;
BIT() : n(0) {}
BIT(int n) {
    n = -n; t.assign(n + 1, 0);}
static int lowbit(int x) {return x & -x;}
T query(int i) const {
    T ans = 0;
    for (; i > 0; i -= lowbit(i)) ans += t[i];
    return ans;}
void upd(int i, T val) {
    if (i <= 0) return;
    for (; i <= n; i += lowbit(i)) t[i] += val;}
T query(int l, int r) const {
    if (l > r) return 0;
    return query(r) - query(l - 1);}
void upd(int l, int r, T val) {
    upd(l, val); upd(r + 1, -val);};
```

4.5 Cartesian Tree

```
ll buildCartesianTree(vll&arr, vll&L, vll&R) {
    ll n = arr.size();
    L.assign(n, -1);
    R.assign(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        int last_popped = -1;
        // Max-Heap logic
        while (!st.empty() && arr[st.top()] < arr[i]) {
            last_popped = st.top();
            st.pop();}
        if (last_popped != -1) L[i] = last_popped;
        if (!st.empty()) R[st.top()] = i;
        st.push(i);}
    int root = -1;
    while (!st.empty()) {
        root = st.top();
        st.pop();}
    return root;}
// max depth reachable from a particular node
ll MaxDepthCalc(ll node, vll&maxdepth, vll&L, vll&R) {
    if (node == -1) return 0;
    return maxdepth[node] = 1 + max(MaxDepthCalc(L[node], maxdepth, L, R),
    MaxDepthCalc(R[node], maxdepth, L, R));}
// Simple recursive function to print the tree
// (In-order)
void printInOrder(int node, const vector<int>& arr,
    const vector<int>& L, const vector<int>& R) {
    if (node == -1) return;
    printInOrder(L[node], arr, L, R);
    cout << arr[node] << " ";
    printInOrder(R[node], arr, L, R);}
const int LOG = 20;
int up[N][LOG], depth[N];
void dfs_lca(int u, int p, int d, const vector<int>&
    L, const vector<int>& R) {
    depth[u] = d;
    up[u][0] = p;
    for (int i = 1; i < LOG; i++)
        up[u][i] = up[up[u][i-1]][i-1];
    if (L[u] != -1) dfs_lca(L[u], u, d + 1, L, R);
    if (R[u] != -1) dfs_lca(R[u], u, d + 1, L, R);}
```

```
//lca
int get_lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    for (int i = LOG - 1; i >= 0; i--) {
        if (depth[u] - (1 << i) >= depth[v]) u =
            up[u][i];}
    if (u == v) return u;
    for (int i = LOG - 1; i >= 0; i--) {
        if (up[u][i] != up[v][i]) {
            u = up[u][i];
            v = up[v][i];}}
    return up[u][0];}
// subtree size calc
int sz[N];
void computeSize(int u, const vector<int>& L, const
    vector<int>& R) {
    if (u == -1) return;
    sz[u] = 1;
    if (L[u] != -1) {
        computeSize(L[u], L, R);
        sz[u] += sz[L[u]];}
    if (R[u] != -1) {
        computeSize(R[u], L, R);
        sz[u] += sz[R[u]];}
    // compute range where arr[i] is the maximum/minimum
    int range_l[N], range_r[N];
    void computeBounds(int u, int l, int r, const
        vector<int>& L, const vector<int>& R) {
        if (u == -1) return;
        range_l[u] = l;
        range_r[u] = r;
        if (L[u] != -1) computeBounds(L[u], l, u - 1, L,
            R);
        if (R[u] != -1) computeBounds(R[u], u + 1, r, L,
            R);}
    // Usage: computeBounds(root, 0, n - 1, L, R);
// tree flattening:
int tin[N], tout[N], timer;
void dfs_euler(int u, const vector<int>& L, const
    vector<int>& R) {
    if (u == -1) return;
    tin[u] = ++timer;
    dfs_euler(L[u], L, R);
    dfs_euler(R[u], L, R);
    tout[u] = timer;}
```

4.6 DSUParity

```
// Description: DSU with parity tracking. Maintains
// distance parity (even/odd) from root.
// Useful for bipartite checking and edge parity
// problems.
// Time:  $\mathcal{O}(\log(N))$  per operation
struct DSUParity {
    vector<int> par, siz, len;
    DSUParity(int n) : par(n + 1), siz(n + 1, 1),
        len(n + 1, 0) {}
    for (int i = 0; i <= n; i++) par[i] = i;}
    pii find(int x) {
        if (par[x] == x) return {x, 0};
        auto px = find(par[x]);
        par[x] = px.first;
        len[x] = (len[x] + px.second) % 2;
        return {par[x], len[x]};}
    void unite(int x, int y) {
        auto px = find(x);
        auto py = find(y);
        if (siz[px.first] < siz[py.first]) swap(px, py);
        par[py.first] = px.first;
        len[py.first] = (py.second + px.second + 1) %
            2;}
```

```
siz[px.first] += siz[py.first];};
```

4.7 DynamicConnectivity

```
vector<pair<ll, ll>> seg[4 * MAX]; // segment tree
// storing edges
ll ans[MAX];
DSU Rollback dsu;
// apply edges active in this segment
void add_edge(int l, int r, pair<ll, ll> e, int x, int
    lx, int rx) {
    if (rx < l || r < lx) return;
    if (lx >= l && rx <= r) {
        seg[x].push_back(e);
        return;}
    int mid = (lx + rx) / 2;
    add_edge(l, r, e, 2 * x + 1, lx, mid);
    add_edge(l, r, e, 2 * x + 2, mid + 1, rx);}
void dfs(int x, int lx, int rx) {
    int snap = dsu.snapshot();
    // apply edges active in this segment
    for (auto &e : seg[x])
        dsu.merge(e.first, e.second);
    if (lx == rx) {
        ans[lx] = dsu.count();
        dsu.rollback_to(snap);
        return;}
    int mid = (lx + rx) / 2;
    dfs(2 * x + 1, lx, mid);
    dfs(2 * x + 2, mid + 1, rx);
    dsu.rollback_to(snap);}
```

4.8 lazyRangeAdd

```
typedef long long ll;
typedef vector<ll> vll;
struct SegTree {
    ll sz = 0;
    vll res;
    vll lazy;
    ll dummy;
    void init(int n, ll d) {
        dummy = d;
        sz = 1;
        while (sz < n) sz *= 2;
        res.assign(sz * 2, d);
        lazy.assign(sz * 2, 0);}
    ll op(ll a, ll b) {
        return a + b;}
    void push(int x, int lx, int rx) {
        if (lazy[x] == 0) return;
        res[x] += (rx - lx + 1) * lazy[x];
        if (lx != rx) {
            lazy[2 * x + 1] += lazy[x];
            lazy[2 * x + 2] += lazy[x];}
        lazy[x] = 0;}
    void build(vll &a, int x, int lx, int rx) {
        lazy[x] = 0;
        if (lx == rx) {
            if (lx < (int)a.size())
                res[x] = a[lx];
            else
                res[x] = dummy;
            return;}
        ll mid = (lx + rx) / 2;
        build(a, 2 * x + 1, lx, mid);}
```

```

    build(a, 2 * x + 2, mid + 1, rx);
    res[x] = op(res[2 * x + 1], res[2 * x + 2]);
void update(int l, int r, ll v, int x, int lx, int rx){
    push(x, lx, rx);
    if (r < lx || rx < l) return;
    if (lx >= l && rx <= r){
        lazy[x] += v;
        push(x, lx, rx);
        return;
    }
    ll mid = (lx + rx) / 2;
    update(l, r, v, 2 * x + 1, lx, mid);
    update(l, r, v, 2 * x + 2, mid + 1, rx);
    res[x] = op(res[2 * x + 1], res[2 * x + 2]);
ll query(int l, int r, int x, int lx, int rx){
    push(x, lx, rx);
    if (r < lx || rx < l) {
        return dummy;
    }
    if (lx >= l && rx <= r){
        return res[x];
    }
    ll mid = (lx + rx) / 2;
    return op(query(l, r, 2 * x + 1, lx, mid),
        query(l, r, 2 * x + 2, mid + 1, rx));
}

```

4.9 LineContainer

```

// Description: Container where you can add lines of
// the form kx+m, and query maximum values at points
// Useful for dynamic programming ('`convex hull
// trick``').
#pragma once
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k <
        o.k; }
    bool operator<(ll x) const { return p < x; }
};
struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y) {
        if (y == end()) return x->p = inf, 0;
        if (x->k == y->k) x->p = x->m > y->m ? inf :
            -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p; }
    void add(ll k, ll m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y)
            = erase(y);
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y)); }
    ll query(ll x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m; }
};

```

4.10 moalgo

```

struct MO{
    struct node{
        ll id, left, right;
    };
    vector<node> queries;
    vector<ll> val, cnt;
    vector<ll> ans;
    ll num, questions, block;
}

```

```

void build(vector<ll> &vec, vector<pair<ll, ll>>
    &q){
    num = vec.size();
    val.resize(num + 1);
    block = sqrt(num);
    // 1-based indexing
    for (ll i = 1; i <= num; i++)
        val[i] = vec[i - 1];
    questions = q.size();
    queries.resize(questions);
    ans.resize(questions);
    cnt = vec;
    sort(cnt.begin(), cnt.end());
    cnt.erase(unique(cnt.begin(), cnt.end()),
        cnt.end());
    map<ll, ll> mp;
    for (ll i = 0; i < cnt.size(); i++)
        mp[cnt[i]] = i;
    for (ll i = 1; i <= num; i++)
        val[i] = mp[val[i]];
    cnt.clear();
    cnt.resize(mp.size());
    for (ll i = 0; i < questions; i++){
        queries[i].id = i;
        queries[i].left = q[i].first;
        queries[i].right = q[i].second; }
    sort(queries.begin(), queries.end(), [&](node
        one, node two){
        ll start1 = one.left / block, start2
            = two.left / block;
        ll end1 = one.right / block, end2 =
            two.right / block;
        if (start1 == start2) return end1 < end2;
        return start1 < start2; }); }

ll times = 0;
void add(ll ind){
    if (!cnt[ind])
        times++;
    cnt[ind]++;
}
void del(ll ind){
    if (cnt[ind] == 1)
        times--;
    cnt[ind]--;
}
vector<ll> getResult(){
    ll start = 1, end = 1;
    add(val[1]);
    for (ll i = 0; i < questions; i++){
        ll left = queries[i].left, right =
            queries[i].right;
        while (end < right)
            add(val[++end]);
        while (start > left)
            add(val[--start]);
        while (end > right)
            del(val[end--]);
        while (start < left)
            del(val[start++]);
        ans[queries[i].id] = times; }
    return ans; }

```

4.11 Persistent Segtree

```

//Your task is to maintain a list of arrays which
// initially has
// a single array. You have to process the following
// types of queries:
// 1. Set the value a in array k to x.
// 2. Calculate the sum of values in range [a,b] in array
// k.
// 3. Create a copy of array k and add it to the end of
// the list.

```

```

#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef vector<ll> vll;
const int MX = 1e6 + 9;
int root[MX];
struct Node{
    int left = 0;
    int right = 0;
    ll val = 0;
};
Node nodes[MX * 40];
struct PerSegTree{
    int id = 0;
    int build(vll &a, int lx, int rx){
        int cur = ++id;
        if (lx == rx){
            nodes[cur].val = a[lx];
            return cur;
        }
        int mid = (lx + rx) / 2;
        nodes[cur].left = build(a, lx, mid);
        nodes[cur].right = build(a, mid + 1, rx);
        nodes[cur].val = nodes[nodes[cur].left].val +
            nodes[nodes[cur].right].val;
        return cur;
    }
    int update(int prev, int i, ll v, int lx, int rx){
        int cur = ++id;
        nodes[cur] = nodes[prev];
        if (lx == rx){
            nodes[cur].val = v;
            return cur;
        }
        int mid = (lx + rx) / 2;
        if (i <= mid)
            nodes[cur].left = update(nodes[cur].left, i,
                v, lx, mid);
        else
            nodes[cur].right = update(nodes[cur].right, i,
                v, mid + 1, rx);
        nodes[cur].val = nodes[nodes[cur].left].val +
            nodes[nodes[cur].right].val;
        return cur;
    }
    ll getSum(int cur, int l, int r, int lx, int rx){
        if (rx < l || r < lx) return 0;
        if (lx >= l && rx <= r) return nodes[cur].val;
        int mid = (lx + rx) / 2;
        return getSum(nodes[cur].left, l, r, lx, mid) +
            getSum(nodes[cur].right, l, r, mid + 1, rx);
    }
    ll kth(int u, int v, int k, int lx, int rx, vector<ll>
        &sortedA){ // 0 based
        if (lx == rx) return sortedA[lx];
        int mid = (lx + rx) / 2;
        int cnt = nodes[nodes[u].left].val -
            nodes[nodes[u].right].val;
        if (k <= cnt)
            return kth(nodes[u].left, nodes[v].left, k,
                lx, mid, sortedA);
        else
            return kth(nodes[u].right, nodes[v].right, k -
                cnt, mid + 1, rx, sortedA);
    }
}
int main() {
    ios base::sync_with_stdio(false);
    cin.tie(NULL);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vll a(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    PerSegTree tree;
    int version_count = 1;
    root[version_count] = tree.build(a, 1, n);
}

```

```

while (q--) {
    int type;
    cin >> type;
    if (type == 1) {
        int k, idx; ll x;
        cin >> k >> idx >> x;
        // Update array k directly
        root[k] = tree.update(root[k], idx, x, 1, n);
    }
    else if (type == 2) {
        int k, l, r;
        cin >> k >> l >> r;
        // Query array k
        cout << tree.getSum(root[k], l, r, 1, n) << "
        \n";
    }
    else if (type == 3) {
        int k;
        cin >> k;
        // Create a new copy referencing array k's
        \ current state
        version count++;
        root[version_count] = root[k];
    }
    return 0;
}

```

4.12 Segment Tree Merge Sort Tree

```

const ll MAXN=3e4+10;
vector<ll> tree[4*MAXN];
void build(ll a[], ll node, ll search_l, ll search_r){
    // root node = 1
    // search_l and search_r is 0-indexed
    if(search_l==search_r){
        tree[node]=vector<ll>(1,a[search_l]);
    }else{
        ll mid=(search_l+search_r)/2;
        build(a, node*2, search_l, mid);
        build(a, node*2+1, mid+1, search_r);
        merge(tree[node*2].begin(), tree[node*2].end(),
        tree[node*2+1].begin(), tree[node*2+1].end(),
        back_inserter(tree[node]));
    }
    ll VectorsValue(vector<ll>& v, ll k){
        // Here goes the algorithm for the value you want
        \ to extract
        ll n=v.size();
        auto it=upper_bound(v.begin(), v.end(), k);
        if(it==v.end()){
            return 0;
        }else{
            ll x=it-v.begin();
            return n-x;
        }
    }
    ll query(ll node, ll search_l, ll search_r, ll l, ll
    \ r, ll val) {
        // root node = 1
        // initial search_l = 0, search_r = 0, l= 0 indexed
        \ left pointer (inclusive)
        // l = 0 indexed left pointer (inclusive)
        // r = 0 indexed right pointer (inclusive)
        if (l>r) return 0;
        if (l==search_l && r==search_r) {
            return VectorsValue(tree[node],val);
        }
        ll mid=(search_l+search_r)/2;
        return query(node*2,search_l,mid,l,min(r,mid),val)
        \ +
        query(node*2+1,mid+1,search_r,max(l,mid+1),
        \ r,val);
    }
}

```

4.13 SegmentTreeTamzid

```

struct segtreeforsubsegmentsum{
    struct data{
        ll sum, pref, suff, ans;
    }
};

```

```

};
ll siz = 1;
vector<data> val;
data make_data(ll val){
    data res;
    res.sum = val;
    res.pref = res.suff = res.ans = max(0LL, val);
    return res;
}
void initialize(ll num){
    while (siz < num)
        siz *= 2;
    val.resize(2 * siz + 1, make_data(0));
}
data merge(data l, data r){
    data res;
    res.sum = l.sum + r.sum;
    res.pref = max(l.pref, l.sum + r.pref);
    res.suff = max(r.suff, l.suff + r.sum);
    res.ans = max({l.ans, r.ans, l.suff + r.pref});
    return res;
}
void build(vector<ll> &vec, ll parent, ll start,
    \ ll end){
    if (end - start == 1){
        if (start < vec.size())
            val[parent] = make_data(vec[start]);
        return;
    }
    ll mid = (start + end) / 2;
    build(vec, 2 * parent + 1, start, mid);
    build(vec, 2 * parent + 2, mid, end);
    val[parent] = merge(val[2 * parent + 1], val[2
    \ * parent + 2]);
}
void build(vector<ll> &vec){
    build(vec, 0, 0, siz);
}
void update(ll value, ll ind, ll parent, ll start,
    \ ll end){
    if (end - start == 1){
        val[parent] = make_data(value);
        return;
    }
    ll mid = (start + end) / 2;
    if (ind < mid)
        update(value, ind, 2 * parent + 1, start,
        \ mid);
    else
        update(value, ind, 2 * parent + 2, mid,
        \ end);
    val[parent] = merge(val[2 * parent + 1], val[2
    \ * parent + 2]);
}
void update(ll val, ll ind){
    update(val, ind, 0, 0, siz);
}
data query(ll lo, ll hi, ll parent, ll start, ll
    \ end){
    if (lo <= start && end - 1 <= hi)
        return val[parent];
    if (start > hi || end <= lo)
        return make_data(0);
    ll mid = (start + end) / 2;
    return merge(query(lo, hi, 2 * parent + 1,
    \ start, mid), query(lo, hi, 2 * parent + 2,
    \ mid, end));
}
ll query(ll lo, ll hi){
    return query(lo, hi, 0, 0, siz).ans;
}

```

4.14 SparseWithIndex

```

const int N = 1e5 + 9;
int a[N];
pair<int, int> t[N][18]; // {value, index}
pair<int, int> combine(pair<int, int> L, pair<int,
    \ int> R) {
    int val = min(L.first, R.first); // change min to
    \ __gcd for GCD table
}

```

```

if (val == L.first && val == R.first)
    return {val, min(L.second, R.second)};
else if (val == L.first) return L;
else if (val == R.first) return R;
else return {val, min(L.second, R.second)};
}
void build(int n) {
    for (int i = 1; i <= n; ++i)
        t[i][0] = {a[i], i};
    for (int k = 1; k < 18; ++k) {
        for (int i = 1; i + (1 <= k) - 1 <= n; ++i) {
            t[i][k] = combine(t[i][k - 1], t[i + (1 <=
            \ k) - 1][k - 1]);
        }
    }
    pair<int, int> query(int l, int r) {
        int k = 31 - __builtin_clz(r - l + 1);
        return combine(t[l][k], t[r - (1 <= k) + 1][k]);
    }
}

```

4.15 Treap

```

struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;
    Node(int val) : val(val), y(rand()) {}
    void recalc();
    int cnt(Node* n) { return n ? n->c : 0; }
    void Node::recalc() { c = cnt(l) + cnt(r) + 1; }
    template<class F> void each(Node* n, F f) {
        if (n) { each(n->l, f); f(n->val); each(n->r, f);
        \ }
    }
    pair<Node*, Node*> split(Node* n, int k) {
        if (!n) return {};
        if (cnt(n->l) >= k) { // "n->val >= k" for
            \ lower_bound(k)
            auto pa = split(n->l, k);
            n->l = pa.second;
            n->recalc();
            return {pa.first, n};
        } else {
            auto pa = split(n->r, k - cnt(n->l) - 1); //
            \ and just "k"
            n->r = pa.first;
            n->recalc();
            return {n, pa.second};
        }
    }
    Node* merge(Node* l, Node* r) {
        if (!l) return r;
        if (!r) return l;
        if (l->y > r->y) {
            l->r = merge(l->r, r);
            l->recalc();
            return l;
        } else {
            r->l = merge(l, r->l);
            r->recalc();
            return r;
        }
    }
    Node* ins(Node* t, Node* n, int pos) {
        auto pa = split(t, pos);
        return merge(merge(pa.first, n), pa.second);
    }
    // Example application: move the range [l, r] to index
    \ k
    void move(Node*& t, int l, int r, int k) {
        Node *a, *b, *c;
        tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
        if (k <= l) t = merge(merge(a, b, k), c);
        else t = merge(a, ins(c, b, k - r));
    }
}

```

4.16 Triebit

```

struct Trie {

```

```

static const int B = 31;
struct node {
    node *nxt[2];
    int sz;
    node() {
        nxt[0] = nxt[1] = NULL;
        sz = 0; } *root;
Trie() { root = new node(); }
void insert(int val) {
    node *cur = root;
    cur->sz++;
    for (int i = B - 1; i >= 0; i--) {
        int b = val >> i & 1;
        if (cur->nxt[b] == NULL)
            cur->nxt[b] = new node();
        cur = cur->nxt[b];
        cur->sz++;
    }
int query(int x, int k) {
    // number of values s.t. val ^ x < k
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
        if (cur == NULL)
            break;
        int b1 = x >> i & 1, b2 = k >> i & 1;
        if (b2 == 1) {
            if (cur->nxt[b1])
                ans += cur->nxt[b1]->sz;
            cur = cur->nxt[!b1];
        } else
            cur = cur->nxt[b1];
    }
    return ans;
int get_max(int x) { // returns maximum of val ^ x
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
        int k = x >> i & 1;
        if (cur->nxt[!k])
            cur = cur->nxt[!k], ans <= 1, ans++;
        else
            cur = cur->nxt[k], ans <= 1;
    }
    return ans;
int get_min(int x) { // returns minimum of val ^ x
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
        int k = x >> i & 1;
        if (cur->nxt[k])
            cur = cur->nxt[k], ans <= 1;
        else
            cur = cur->nxt[!k], ans <= 1, ans++;
    }
    return ans;
void del(node *cur) {
    for (int i = 0; i < 2; i++)
        if (cur->nxt[i])
            del(cur->nxt[i]);
    delete (cur);
}
// Next Combination Mask
int next_combs_mask(int mask) {
    int lsb = -mask & mask;
    return ((mask + lsb) ^ mask) / (lsb << 2) |
        (mask + lsb);
}
// Iterate over submask in decreasing order
for (int s = mask; s > 0; s = (s - 1) & mask) {}

```

5 Dynamic Programming

5.1 CHT

```

struct line{
    ll m, c;
    line(ll a, ll b){

```

```

        m = a, c = b;}};
//1. mi > mi+1, minimum.
//2. mi > mi+1, maximum.
//3. mi < mi+1, minimum.
//4. mi < mi+1, maximum.
struct CHT{
    vector<line> v;
    int t, ptr;
    void init(int tp){
        t = tp, ptr = 0, v.clear();
    }
    CHT(int tp){
        t = tp, ptr = 0;
    }
    bool bad(line l1, line l2, line l3){
        __int128 a = (__int128)(l3.c - l1.c) * (l1.m -
            l2.m);
        __int128 b = (__int128)(l2.c - l1.c) * (l1.m -
            l3.m);
        if (t == 1 or t == 4)
            return a <= b;
        return a >= b;
    }
    void add(line a){
        v.push_back(a);
        int sz = v.size();
        while (sz >= 3 && bad(v[sz - 3], v[sz - 2],
            v[sz - 1])){
            v.erase(v.end() - 2, sz--);
        }
    }
    inline ll val(int ind, ll x){
        return v[ind].m * x + v[ind].c;
    }
    ll query1(ll x) {
        // ternary search
        int l = 0, r = v.size() - 1;
        ll ans = 0;
        while (l <= r){
            int mid1 = l + (r - l) / 3, mid2 = r - (r - l) / 3;
            if (t & 1){
                if (val(mid1, x) <= val(mid2, x))
                    r = mid2 - 1, ans = val(mid1, x);
                else
                    l = mid1 + 1, ans = val(mid2, x);
            }
            else{
                if (val(mid1, x) >= val(mid2, x))
                    r = mid2 - 1, ans = val(mid1, x);
                else
                    l = mid1 + 1, ans = val(mid2, x);
            }
        }
        return ans;
    }
    ll query2(ll x) {
        // 1,4 if xi<=xi+1; 2,3 if xi>=xi+1
        if (v.empty())
            return 0;
        if (ptr >= v.size())
            ptr = v.size() - 1;
        while (ptr < v.size() - 1){
            if (t & 1){
                if (val(ptr, x) > val(ptr + 1, x))
                    ptr++;
                else break;
            }
            else{
                if (val(ptr, x) < val(ptr + 1, x))
                    ptr++;
                else break;
            }
        }
        return val(ptr, x);
    }
}

```

5.2 DivideNConquer

```

ll num, k; cin >> num >> k;
vector<ll> vec(num); for (auto &it : vec) cin >> it;
vector<ll> pref(num + 2), weightedpref(num + 2);
for (ll i = 1; i <= num; i++){
    pref[i] = vec[i - 1] + pref[i - 1];
    weightedpref[i] = vec[i - 1] * i + weightedpref[i - 1];
}

```

```

function<ll(ll, ll)> costfunc = [&](ll start, ll end){
    return (pref[end] - pref[start - 1]) * (pref[end]
        - pref[start - 1]);
}
vector<vector<ll>> ans(k + 1, vector<ll>(num + 1,
    INF));
function<void(ll, ll, ll, ll, ll)> go = [&](ll times,
    ll start, ll end, ll lo, ll hi){
    if (lo > hi) return;
    ll mid = (lo + hi + 1) / 2, bestval = INF, bestpos
        = start;
    for (ll i = start; i <= min(mid, end); i++){
        ll now = ans[times - 1][i - 1] + costfunc(i,
            mid);
        if (now < bestval){
            ans[times][mid] = now;
            bestval = now;
            bestpos = i;
        }
        go(times, start, bestpos, lo, mid - 1);
        go(times, bestpos, end, mid + 1, hi);
    }
    for (ll i = 1; i <= num; i++){
        ans[1][i] = costfunc(1, i);
    }
    for (ll i = 2; i <= k; i++) go(i, 1, num, 1, num);
    cout << ans[k][num] << endl;
}

```

5.3 LiChaoTree

```

struct Line{
    ll m, c;
    Line(ll m=0, ll c=INF):m(_m),c(_c){}
    ll eval(ll x) const{
        return (ll)((__int128)m*x+c);
    }
}
struct Node{
    Line line; Node *left, *right;
    Node(Line v){
        line=v; left=right=NULL;
    }
    ~Node() {
        delete left; delete right;
    }
}
struct LiChao{
    ll L, R;
    Node* root;
    LiChao(ll l, ll r):L(l),R(r){
        root=new Node(Line());
    }
    void insert_line(Line nw, ll lx, ll rx, Node* node){
        ll mid=(lx+rx)>>1;
        bool left=nw.eval(lx)<node->line.eval(lx);
        bool midp=nw.eval(mid)<node->line.eval(mid);
        if (midp) swap(nw, node->line);
        if (lx==rx) return;
        if (left!=midp){
            if (!node->left) node->left=new Node(Line());
            insert_line(nw, lx, mid, node->left);
        }
        else{
            if (!node->right) node->right=new Node(Line());
            insert_line(nw, mid+1, rx, node->right);
        }
    }
    void insert_segment(Line nw, ll ql, ll qr, ll lx, ll
        rx, Node* node){
        if (qr<lx || rx<ql) return;
        if (ql==lx && rx==qr){
            insert_line(nw, lx, rx, node);
            return;
        }
        ll mid=(lx+rx)>>1;
        if (!node->left) node->left=new Node(Line());
        if (!node->right) node->right=new Node(Line());
        insert_segment(nw, ql, qr, lx, mid, node->left);
        insert_segment(nw, ql, qr, mid+1, rx, node->right);
    }
    ll query_point(ll x, ll lx, ll rx, Node* node){
        if (!node) return INF;
        ll res=node->line.eval(x);
        if (lx==rx) return res;
    }
}

```

```

ll mid=(lx+rx)>>1;
if(x<=mid&&node->left)
    return
    min(res,query_point(x,lx,mid,node->left));
if(x>mid&&node->right)
    return min(res,query_point(x,mid+1,rx,node->ri
    ght));
return res;
~LiChao() {
    delete root;
};
int main(){
    LiChao lc(-1000000,1000000);
    lc.insert_line(Line(2,3),lc.L,lc.R,lc.root);
    lc.insert_line(Line(-1,10),lc.L,lc.R,lc.root);
    lc.insert_segment(Line(5,0),-5,5,lc.L,lc.R,lc.root
    );
    for(int x=-5;x<=5;x++){
        cout<<"x = "<<x<<" -> min y = "
        <<lc.query_point(x,lc.L,lc.R,lc.root)<<"
        <<" ";
    }
    return 0;}

```

5.4 SOSDP

```

const ll Mlog = 20;
const ll maxn = (1LL << Mlog);
void forward1(ll dp[]) { //subsets are added in
    supersets
    for(ll bit = 0; bit < Mlog; bit++){
        for(ll mask = 0; mask < maxn; mask++){
            if(mask & (1 << bit)){
                dp[mask] += dp[mask ^ (1 << bit)];
                dp[mask] %= M;}}}
void forward2(ll dp[]) { //supersets are added in
    subsets
    for(ll bit = 0; bit < Mlog; bit++){
        for(ll mask = maxn - 1; mask >= 0; mask--){
            if(mask & (1LL << bit)){
                dp[mask ^ (1LL << bit)] += dp[mask];
                dp[mask ^ (1LL << bit)] %= M;}}}
// how many non-empty subsets of these numbers have a
    union of bits that exactly matches a specified
    binary target
forward1(freq1);
for(int i = 0; i < maxn; i++){
    ll numsets = bigmod(2, freq1[i]) % M;
    freq1[i] = (numsets - 1 + M) % M;}
backward1(freq1); // Reverse inner loop of forward1 and
    subtract instead of adding
cout << freq1[req] << "" ;

```

6 Geometry

6.1 3D

6.1.1 3D Convex Hull

```

#define vi vector<int>
#define pii pair<int, int>
#define rep(i, a, b) for(int i = (a); i < (b); i++)
template<typename T>
using minpq = priority_queue<T, vector<T>, greater<T>>;
typedef long double ftype;
struct pt3 {
    ftype x, y, z;
    pt3(ftype x = 0, ftype y = 0, ftype z = 0) : x(x),
        y(y), z(z) {}
    pt3 operator-(const pt3 &o) const {
        return pt3(x - o.x, y - o.y, z - o.z);}

```

```

pt3 cross(const pt3 &o) const {
    return pt3(y * o.z - z * o.y, z * o.x - x *
        o.z, x * o.y - y * o.x);}
ftype dot(const pt3 &o) const {
    return x * o.x + y * o.y + z * o.z;}};
struct face {
    int a, b, c;
    pt3 q;};
const ftype EPS = 1e-9;
vector<face> hull3(const vector<pt3> &p) {
    int n = sz(p);
    assert(n >= 3);
    vector<face> f;
    vector<vector<bool>> dead(n, vector<bool>(n,
        true));
    auto add_face = [&](int a, int b, int c) {
        f.push_back({a, b, c, (p[b] - p[a]).cross(p[c]
            - p[a])});
        dead[a][b] = dead[b][c] = dead[c][a] = false;};
    add_face(0, 1, 2);
    add_face(0, 2, 1);
    rep(i, 3, n) {
        vector<face> f2;
        for(face &F : f) {
            if((p[i] - p[F.a]).dot(F.q) > EPS) {
                dead[F.a][F.b] = dead[F.b][F.c] =
                    dead[F.c][F.a] = true;
            }
            f2.push_back(F);
        }
        f.clear();
        for(face &F : f2) {
            int arr[3] = {F.a, F.b, F.c};
            rep(j, 0, 3) {
                int a = arr[j], b = arr[(j + 1) % 3];
                if(dead[b][a]) {
                    add_face(b, a, i);}}}
        f.insert(f.end(), all(f2));}
    return f;
}

```

6.2 Circle

6.2.1 Circle Intersection

Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

```

#include "Point.h"
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P,
    P>* out) {
    if (a == b) { assert(r1 != r2); return false; }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
        p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 -
            p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0,
        h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}

```

6.2.2 Circle Polygon Intersection

Returns the area of the intersection of a circle with a ccw polygon. Time: $O(n)$

```

#include "Point.h"
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b =
            (p.dist2()-r*r)/d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1.,
            -a+sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;
        return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) *
            r2;
    };
    auto sum = 0.0;
    rep(i,0,sz(ps))
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}

```

6.2.3 Circle Tangents

Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

```

#include "Point.h"
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2,
    double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr *
        dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}

```

6.3 Polygon

6.3.1 Hull Diameter

Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

```
#include "Point.h"
```

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i, 0, j)
        for (; j = (j + 1) % n) {
            res = max(res, {(S[i] - S[j]).dist2(), {S[i],
                S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] -
                S[i]) >= 0)
                break;
        }
    return res.second;
}
```

6.3.2 Line Hull Intersection

Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon:

- $(-1, -1)$ if no collision, • $(i, -1)$ if touching the corner i ,
- (i, i) if along side $(i, i + 1)$, • (i, j) if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line. Time: $O(\log n)$

```
#include "Point.h"
```

```
#define cmp(i, j)
    ~ sgn(dir.perp()).cross(poly[(i)%n] - poly[(j)%n])
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n)
    < 0
template <class P> int extrVertex(vector<P>& poly, P
    ~ dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi :
            ~ lo) = m;
    }
    return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i, 0, 2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
}
```

```
if (res[0] == res[1]) return {res[0], -1};
if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) %
        ~ sz(poly)) {
        case 0: return {res[0], res[0]};
        case 2: return {res[1], res[1]};
    }
return res;
```

6.3.3 Polygon Center

Returns the center of mass for a polygon.
Time: $O(n)$

```
#include "Point.h"
```

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

6.4 Closest_Pair

```
#include <bits/stdc++.h>
using namespace std;
const int N = 3e5 + 9;
#define x first
#define y second
long long dist2(pair<int, int> a, pair<int, int> b) {
    return 1LL * (a.x - b.x) * (a.x - b.x) + 1LL *
        ~ (a.y - b.y) * (a.y - b.y);
}
pair<int, int> closest_pair(vector<pair<int, int>> a) {
    int n = a.size();
    assert(n >= 2);
    vector<pair<int, int>, int> p(n);
    for (int i = 0; i < n; i++) p[i] = {a[i], i};
    sort(p.begin(), p.end());
    int l = 0, r = 2;
    long long ans = dist2(p[0].x, p[1].x);
    pair<int, int> ret = {p[0].y, p[1].y};
    while (r < n) {
        while (l < r && 1LL * (p[r].x.x - p[l].x.x) *
            ~ (p[r].x.x - p[l].x.x) >= ans) l++;
        for (int i = l; i < r; i++) {
            long long nw = dist2(p[i].x, p[r].x);
            if (nw < ans) {
                ans = nw;
                ret = {p[i].y, p[r].y};
            }
            r++;
        }
        return ret;
    }
}
int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int n; cin >> n;
    vector<pair<int, int>> p(n);
    for (int i = 0; i < n; i++) cin >> p[i].x >>
        ~ p[i].y;
    pair<int, int> z = closest_pair(p);
    if (z.x > z.y) swap(z.x, z.y);
    cout << z.x << ' ' << z.y << ' ' << fixed <<
        ~ setprecision(6) << sqrtl(dist2(p[z.x],
            ~ p[z.y])) << '\n';
    return 0;
}
```

6.5 Half Plane

```
// contains all points p such that: cross(b - a, p - a)
    ~ >= 0
struct HP {
    pt a, b;
    HP() {}
    HP(pt a, pt b) : a(a), b(b) {}
    HP(const HP& rhs) : a(rhs.a), b(rhs.b) {}
    int operator < (const HP& rhs) const {
        pt p = b - a;
        pt q = rhs.b - rhs.a;
        int fp = (p.y < 0 || (p.y == 0 && p.x < 0));
        int fq = (q.y < 0 || (q.y == 0 && q.x < 0));
        if (fp != fq) return fp == 0;
        if (cross(p, q)) return cross(p, q) > 0;
        return cross(p, rhs.b - a) < 0;
    }
    pt line_line_intersection(pt a, pt b, pt c, pt d) {
        b = b - a; d = d - c; c = c - a;
        return a + b * cross(c, d) / cross(b, d);
    }
    pt intersection(const HP& v) {
        return line_line_intersection(a, b, v.a, v.b);
    }
};
int check(HP a, HP b, HP c) {
    return cross(a.b - a.a, b.intersection(c) - a.a) >
        ~ -eps;
// -eps to include polygons of zero area (straight
    ~ lines, points)}
// consider half-plane of counter-clockwise side of
    ~ each line
// if lines are not bounded add infinity rectangle
// returns a convex polygon, a point can occur multiple
    ~ times though
// complexity:  $O(n \log(n))$ 
vector<pt> half_plane_intersection(vector<HP> h) {
    sort(h.begin(), h.end());
    vector<HP> tmp;
    for (int i = 0; i < h.size(); i++) {
        if (!i || cross(h[i].b - h[i].a, h[i - 1].b -
            ~ h[i - 1].a)) {
            tmp.push_back(h[i]);
        }
        h = tmp;
        vector<HP> q(h.size() + 10);
        int qh = 0, qe = 0;
        for (int i = 0; i < h.size(); i++) {
            while (qh - qe > 1 && !check(h[i], q[qh - 2],
                ~ q[qh - 1])) qe--;
            while (qh - qe > 1 && !check(h[i], q[qh], q[qh
                ~ + 1])) qh++;
            q[qh++] = h[i];
        }
        while (qh - qe > 2 && !check(q[qh], q[qh - 2],
            ~ q[qh - 1])) qe--;
        while (qh - qe > 2 && !check(q[qh - 1], q[qh],
            ~ q[qh + 1])) qh++;
        vector<HP> res;
        for (int i = qh; i < qe; i++) res.push_back(q[i]);
        vector<pt> hull;
        if (res.size() > 2) {
            for (int i = 0; i < res.size(); i++) {
                hull.push_back(res[i].intersection(res[(i
                    ~ + 1) % ((int)res.size())]));
            }
            return hull;
        }
    }
```

6.6 inside_poly_fast

```
vector<pt> seq;
pt translation; int n;
//int sgn(long long val) { return val > 0 ? 1 : (val ==
    0 ? 0 : -1); }
bool lexComp(const pt &l, const pt &r) {
    if (sign(l.x - r.x) != 0) return l.x < r.x;
    return l.y < r.y; }
bool pointInTriangle(pt a, pt b, pt c, pt point) {
    T s1 = abs(cross3(a, b, c));
    T s2 = abs(cross3(point, a, b)) +
        abs(cross3(point, b, c)) + abs(cross3(point,
            c, a));
    return sign(s1 - s2) == 0; }
void prepare(vector<pt> &points) {
    n = points.size();
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (lexComp(points[i], points[pos])) pos = i; }
    rotate(points.begin(), points.begin() + pos,
        points.end());
    n--; seq.resize(n);
    for (int i = 0; i < n; i++)
        seq[i] = points[i + 1] - points[0];
    translation = points[0]; }
bool pointInConvexPolygon(pt point) {
    point = point - translation;
    if (sign(cross(seq[0], point)) != 0 &&
        sign(cross(seq[0], point)) != sign(cross(seq[0],
            seq[n - 1])))
        return false;
    if (sign(cross(seq[n - 1], point)) != 0 &&
        sign(cross(seq[n - 1], point)) != sign(cross(seq[n
            - 1], seq[0])))
        return false;
    if (sign(cross(seq[0], point)) == 0)
        return NORM(seq[0]) >= NORM(point);
    int l = 0, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        int pos = mid;
        if (sign(cross(seq[pos], point)) >= 0) l = mid;
        else r = mid; }
    int pos = l;
    return pointInTriangle(seq[pos], seq[pos + 1],
        pt(0, 0), point); }
```

6.7 Manhattan MST

```
//Returns {weight of mst, edges of mst} for the set of
// 2d points
//O(n log n)
template<typename T, typename C>
pair<C, vector<array<int, 2>>>
    manhattan_mst_2d(vector<array<T, 2>> m) {
    vector<tuple<T, int, int>> candidates;
    candidates.reserve(m.size() * 4);
    vector<int> id(m.size()); iota(id.begin(), id.end(),
        0);
    for (int rot = 0; rot < 4; ++rot) {
        sort(id.begin(), id.end(), [&](const auto& l,
            const auto& r) { return m[l][0] + m[l][1] <
                m[r][0] + m[r][1]; });
        map<int, int, greater<int>> active;
        for (auto i : id) {
            for (auto it = active.lower_bound(m[i][0]); it
                != active.end(); active.erase(it++)) {
                int j = it->second; }
```

```
        if (m[i][0] - m[i][1] > m[j][0] - m[j][1])
            break;
        candidates.push_back({abs(m[i][0] -
            m[j][0]) + abs(m[i][1] - m[j][1]), i,
            j});
        active[m[i][0]] = i;
        for (auto& p : m) rot & 1 ? void(p[0] *= -1) :
            swap(p[0], p[1]);
        sort(candidates.begin(), candidates.end());
        vector<int> pr(m.size(), -1);
        function<int(int)> find = [&](int x) { return pr[x] < 0
            ? x : pr[x] = find(pr[x]); };
        auto unite = [&](int x, int y) {
            if (x = find(x), y = find(y); x == y) return false;
            if (pr[x] < pr[y]) swap(x, y);
            pr[y] += pr[x]; pr[x] = y;
            return true;
        };
        C mst_weight = 0;
        vector<array<int, 2>> mst_edges(m.size() - 1);
        int i = 0;
        for (auto [w, u, v] : candidates) {
            if (unite(u, v)) {
                mst_weight += w;
                mst_edges[i++] = {u, v}; }
        }
        return {mst_weight, mst_edges}; }
```

6.8 MEC

```
// Given n points, return the radius of the minimum
// enclosing circle
// Call convex_hull() before this for faster solution
// Expected O(n)
double minimum_enclosing_circle_radius(vector<pointdb>
    &p) {
    random_shuffle(p.begin(), p.end());
    int n = p.size();
    pointdb center = p[0]; double radius = 0;
    for (int i = 1; i < n; i++) {
        if (DIST(center, p[i]) > radius + EPS) {
            center = p[i]; radius = 0;
            for (int j = 0; j < i; j++) {
                if (DIST(center, p[j]) > radius + EPS) {
                    center = (p[i] + p[j]) / 2.0;
                    radius = DIST(p[i], p[j]) / 2.0;
                    for (int k = 0; k < j; k++) {
                        if (DIST(center, p[k]) > radius + EPS) {
                            pointdb a = p[i], b = p[j], c = p[k];
                            pointdb ab = (a + b) / 2.0, ac = (a + c) / 2.0;
                            pointdb dab = (b - a) * pointdb(0, 1);
                            pointdb dac = (c - a) * pointdb(0, 1);
                            double t = ((ac - ab).x * dab.y - (ac - ab).y
                                * dab.x) /
                                (dab.x * dab.y - dab.y * dab.x);
                            center = ab + dab * t;
                            radius = DIST(center, a); } } } }
        return radius; }
```

6.9 Minkowski

```
// a and b are convex polygons
// returns a convex hull of their minkowski sum
// min(a.size(), b.size()) >= 2
// https://cp-algorithms.com/geometry/minkowski.html
vector<pt> minkowski_sum(vector<pt> a, vector<pt> b) {
    reorder_polygon(a); reorder_polygon(b);
    int n = a.size(), m = b.size();
    int i = 0, j = 0;
    a.push_back(a[0]); a.push_back(a[1]);
    b.push_back(b[0]); b.push_back(b[1]); }
```

```
vector<pt> c;
while (i < n || j < m) {
    c.push_back(a[i] + b[j]);
    double p = cross(a[i + 1] - a[i], b[j + 1] -
        b[j]);
    if (sign(p) >= 0) ++i;
    if (sign(p) <= 0) ++j; }
return c; }
```

6.10 Point 2D

```
#define x real()
#define y imag()
const double PI = acos(-1);
const double eps = 1e-9;
typedef double T;
typedef complex<T> pt;
T sign(double x) { return (x > eps) - (x < -eps); }
T dot(pt a, pt b) { return (conj(a) * b).x; }
T cross(pt a, pt b) { return (conj(a) * b).y; }
T NORM(pt a) { return dot(a, a); }
double NORM2(pt a) { return sqrt(dot(a, a)); }
T dist_sq(pt a, pt b) { return norm(a - b); }
double dist(pt a, pt b) { return abs(a - b); }
double angle_of_elev(pt a, pt b) { return arg(b - a); }
double slope(pt a, pt b) { return tan(arg(b - a)); }
// for polar to cartesian : polar(r, theta)
// for cartesian to polar : pt(abs(p), arg(p))
T cross3(pt a, pt b, pt c) { return cross(b - a, c - a); }
// AB X AC
pt rotate_origin(pt a, double theta) {
    return a * pt(polar(1.0, theta)); }
int orient(pt a, pt b, pt c) {
    double cr = cross(b - a, c - a);
    return sign(cr); // Returns 1 if > eps, -1 if <
        -eps, 0 otherwise }
double angle(pt v, pt w) {
    double cosTheta = (double)dot(v, w) / (NORM2(v) *
        NORM2(w));
    return acos(max(-1.0, min(1.0, cosTheta))); }
double rad_to_deg(double r) { return r * 180.0 / PI; }
double deg_to_rad(double d) { return d * PI / 180.0; }
// p is in the circle with diameter AB
bool inDisk(pt a, pt b, pt p) { return dot(a - p, b - p) <= 0; }
bool onSegment(pt a, pt b, pt p) { return orient(a, b, p) == 0
    && inDisk(a, b, p); }
double areaPolygon(vector<pt> &p) {
    double area = 0.0;
    for (int i = 0, n = p.size(); i < n; i++) {
        area += cross(p[i], p[(i + 1) % n]); }
    return abs(area) / 2.0; }
bool inPolygon(vector<pt> p, pt a, bool strict = true) {
    auto above = [](pt aa, pt p) { return p.y >= aa.y; };
    auto crossesRay = [&](pt a, pt p, pt q) {
        return (above(a, q) - above(a, p)) *
            orient(a, p, q) > 0;
    };
    int numCrossings = 0;
    for (int i = 0, n = p.size(); i < n; i++) {
        if (onSegment(p[i], p[(i + 1) % n], a)) return
            !strict;
        numCrossings += crossesRay(a, p[i], p[(i + 1)
            % n]); }
    return numCrossings & 1; }
bool inAngle(pt a, pt b, pt c, pt p) { // p lies in the angle
    < BAC (CW)
    assert(orient(a, b, c) != 0);
    if (orient(a, b, c) < 0) swap(b, c);
    return orient(a, b, p) >= 0 && orient(a, c, p) <= 0; }
double orientedAngle(pt a, pt b, pt c) { // CW BAC }
```

```

    if(orient(a,b,c)>=0) return angle(b-a,c-a);
    else return 2.0*PI-angle(b-a,c-a);}
bool isConvex(vector<pt>&p){//collection of points
    ~ makes a convex hull
    if(p.size()<3) return false;
    bool hasPos=false,hasNeg=false;
    for (int i=0,n=p.size();i<n;i++){
        int o=orient(p[i],p[(i+1)%n],p[(i+2)%n]);
        if(o>0)hasPos = true;
        if(o<0)hasNeg = true;
    }
    return !(hasPos && hasNeg);}
void convex_hull(vector<pt>& a, bool incl_col=0) {
    if (a.size() <= 1) return;
    pt p0 = *min_element(a.begin(),a.end(),[(pt a,pt
        b){
            return make_pair(a.y,a.x)<make_pair(b.y,b.x);
        }]);
    sort(a.begin(), a.end(), [&p0](const pt&a,const
        pt&b){
        int o=orient(p0,a,b);
        if (o==0)
            return (p0.x-a.x)*(p0.x-a.x) +
                (p0.y-a.y)*(p0.y-a.y)
            < (p0.x-b.x)*(p0.x-b.x) +
                (p0.y-b.y)*(p0.y-b.y);
        return o > 0; // Fixed: Now sorts in CCW order
    });
    if (incl_col){
        int i=(int)a.size()-1;
        while (i>=0 && orient(p0,a[i],a.back())==0)
            i--;
        reverse(a.begin()+i+1,a.end());
    }
    auto ccw=[&](pt a, pt b, pt c,bool incl_col) {
        int o = orient(a, b, c);
        return o > 0 || (incl_col && o == 0);
    };
    vector<pt> st;
    for (int i = 0; i < (int)a.size(); i++) {
        while (st.size() > 1 && !ccw(st[st.size()-2],
            st.back(), a[i], incl_col))
            st.pop_back();
        st.push_back(a[i]);
    }
    if (incl_col == false && st.size() == 2 && st[0]
        == st[1]) st.pop_back();
    a = st;
}
pt perp(pt a) {return pt{-a.y,a.x};}
struct line {
    pt v;T c;
    line(pt v, T c) : v(v), c(c) {}
    line(T a, T b, T c) : v({b, -a}), c(c) {}
    line(pt p, pt q) : v(q - p), c(cross(v, p)) {}
    T side(pt p) const {return cross(v, p) - c;}
    double dist(pt p) const {return fabs(side(p)) /
        NORM2(v);}
    double sqDist(pt p) const {return (double)side(p)
        * side(p) / NORM(v);}
    line perp_through(pt p) const {return {p, p +
        perp(v)};}
    bool cmpProj(pt p, pt q) const {return dot(v, p) <
        dot(v, q);}
    static bool intersection(line l1, line l2, pt
        &out) {
        T d = cross(l1.v, l2.v);
        if (fabs(d) <= eps) return false;
        out = (l2.v * l1.c - l1.v * l2.c) / d;
        return true;
    }
    void shiftLeft(double d){c += d * NORM2(v);}
    pt proj(pt p) const {return p - perp(v) * side(p)
        / NORM(v);}
}

```

```

pt refl(pt p) const {return p - perp(v) * (T)2 *
    side(p) / NORM(v);}
}

```

6.11 Point 3D

```

struct p3{
    T X, Y, Z;
    p3() : X(0), Y(0), Z(0) {}
    p3(T a,T b,T c):X(a),Y(b),Z(c){}
    p3 operator+(p3 p){return {X+p.X,Y+p.Y,Z+p.Z};}
    p3 operator-(p3 p){return {X-p.X,Y-p.Y,Z-p.Z};}
    p3 operator*(T d){return {X*d,Y*d,Z*d};}
    p3 operator/(T d){return {X/d,Y/d,Z/d};}
    bool operator==(p3 p){return
        tie(X,Y,Z)==tie(p.X,p.Y,p.Z);}
    bool operator!=(p3 p){return !operator==(p);}
    friend istream& operator>>(istream&in,p3
        &p){return in>>p.X>>p.Y>>p.Z;}
    friend ostream& operator<<(ostream&os,p3 p){return
        os<<" "<<p.X<<" "<<p.Y<<" "<<p.Z<<" ";}
    friend T operator|(p3 v,p3 w){return
        v.X*w.X+v.Y*w.Y+v.Z*w.Z;}
    friend p3 operator*(p3 v, p3 w) {
        return {v.Y*w.Z-v.Z*w.Y,
            v.Z*w.X-v.X*w.Z,
            v.X*w.Y-v.Y*w.X};};
    bool eq(double a, double b = 0) {return fabs(a - b) <=
        eps;}
    bool isZero(p3 v) {return abs(v) <= eps;}
    T sq(p3 v) {return v|v;}
    double abs(p3 v) {return sqrt(sq(v));}
    p3 unit(p3 v) {return v/abs(v);}
    double angle(p3 v, p3 w) {
        double cosTheta=(v|w)/abs(v)/abs(w);
        return acos(max(-1.0,min(1.0,cosTheta)));}
    T orient(p3 p,p3 q,p3 r,p3 s){
        return (q-p)*(r-p)|(s-p);//+ve -> s above pqr
    }
    T orientByNormal(p3 p, p3 q, p3 r, p3 n){return
        -(q-p)*(r-p)|n;}
    struct plane {
        p3 n; T d;
        // From normal n and offset d
        plane(p3 n, T d) : n(n), d(d) {}
        // From normal n and point P
        plane(p3 n, p3 p) : n(n), d(n|p) {}
        // From three non-collinear points P,Q,R
        plane(p3 p, p3 q, p3 r) : plane((q-p)*(r-p), p) {}
        // Will be defined later:
        // - these work with T = int
        T side(p3 p) {return n|p-d;}
        double dist(p3 p) {return abs(side(p))/abs(n);}
        plane translate(p3 t) {return {n, d+(n|t)};}
        // - these require T = double
        plane shiftUp(double dist) {return {n, d +
            dist*abs(n)};}
        p3 proj(p3 p) {return p - n*side(p)/sq(n);}
        p3 refl(p3 p) {return p - n*2*side(p)/sq(n);}
    };
    struct line3d {
        p3 d, o;
        line3d(p3 p, p3 q) : d(q-p), o(p) {}
        // From two planes p1, p2 (requires T = double)
        line3d(plane p1, plane p2) {
            d = p1.n*p2.n;
            o = (p2.n*p1.d - p1.n*p2.d)*d/sq(d);}
        double sqDist(p3 p) {return sq(d*(p-o))/sq(d);}
        double dist(p3 p) {return sqrt(sqDist(p));}
        bool cmpProj(p3 p, p3 q) {return d|p < d|q;}
        // - these require T = double
        p3 proj(p3 p) {return o + d*(d|(p-o))/sq(d);}
        p3 refl(p3 p) {return proj(p)*2 - p;}
    }
}

```

```

    p3 inter(plane p) {return o - d*p.side(o)/(p.n|d);}
};
p3 zero{0,0,0};
double dist(line3d l1, line3d l2) {
    p3 n = l1.d*l2.d;
    if (n == zero) // parallel
        return l1.dist(l2.o);
    return abs((l2.o-l1.o)|n)/abs(n);}
double smallAngle(p3 v, p3 w) {
    return acos(min(abs(v|w)/abs(v)/abs(w), 1.0));}
double angle(plane p1, plane p2) {
    return smallAngle(p1.n, p2.n);}
bool isParallel(plane p1, plane p2) {
    return p1.n*p2.n == zero;}
bool isPerpendicular(plane p1, plane p2) {
    return (p1.n|p2.n) == 0;}
double angle(line3d l1, line3d l2) {
    return smallAngle(l1.d, l2.d);}
bool isParallel(line3d l1, line3d l2) {
    return l1.d*l2.d == zero;}
bool isPerpendicular(line3d l1, line3d l2) {
    return (l1.d|l2.d) == 0;}
double angle(plane p, line3d l) {
    return M_PI/2 - smallAngle(p.n, l.d);}
bool isParallel(plane p, line3d l) {
    return (p.n|l.d) == 0;}
bool isPerpendicular(plane p, line3d l) {
    return p.n*l.d == zero;}
// centroid of a (possibly non-convex) polygon,
// assuming that the coordinates are listed in a
~ clockwise or
// counterclockwise fashion. Note that the centroid is
~ often known as
// the "center of gravity" or "center of mass".
pt centroid(vector<pt> &p) {
    int n = p.size(); pt c(0, 0);
    double sum = 0;
    for (int i = 0; i < n; i++) sum += cross(p[i],
        p[(i + 1) % n]);
    double scale = 3.0 * sum;
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        c = c + (p[i] + p[j]) * cross(p[i], p[j]);}
    return c / scale;}
line3d perpThrough(plane p, p3 o) {return line3d(o,
    p3(o+p.n));}
plane perpThrough(line3d l, p3 o) {return plane(l.d,
    o);}
//Polygon surface area
p3 vectorArea2(vector<p3> p){
    // vector area * 2 (to avoid divisions)
    p3 S = zero;
    for (int i = 0, n = p.size(); i < n; i++)
        S = S + p[i] * p[(i + 1) % n];
    return S;}
double area(vector<p3> p){
    return abs(vectorArea2(p)) / 2.0;}
double volume(vector<vector<p3>> fs) {
    double vol6 = 0.0;
    for (vector<p3> f : fs)
        vol6 += (vectorArea2(f)|f[0]);
    return abs(vol6) / 6.0;}
}

```

6.12 Polygon Union

```

// calculates the area of the union of n polygons (not
~ necessarily convex).
// the points within each polygon must be given in CCW
~ order.
// complexity: O(N^2), where N is the total number of
~ points
double rat(pt a, pt b, pt p) {

```

```

return !sign(a.x - b.x) ?
(p.y - a.y) / (b.y - a.y) :
(p.x - a.x) / (b.x - a.x);
};
double polygon_union(vector<vector<pt>> &p) {
int n = p.size();
double ans=0;
for(int i = 0; i < n; ++i) {
for (int v = 0; v < (int)p[i].size(); ++v) {
pt a = p[i][v], b = p[i][(v + 1) % p[i].size()];
vector<pair<double, int>> segs;
segs.emplace_back(0, 0), segs.emplace_back(1, 0);
for(int j = 0; j < n; ++j) {
if(i != j) {
for(size_t u = 0; u < p[j].size(); ++u) {
pt c = p[j][u], d = p[j][(u + 1) % p[j].size()];
int sc = sign(cross(b - a, c - a)), sd =
sign(cross(b - a, d - a));
if(!sc && !sd) {
if(sign(dot(b - a, d - c)) > 0 && i > j) {
segs.emplace_back(rat(a, b, c), 1),
segs.emplace_back(rat(a, b, d), -1);}}
else {
double sa = cross(d - c, a - c), sb = cross(d
- c, b - c);
if(sc >= 0 && sd < 0) segs.emplace_back(sa /
(sa - sb), 1);
else if(sc < 0 && sd >= 0)
segs.emplace_back(sa / (sa - sb), -1);}}}}
sort(segs.begin(), segs.end());
double pre = min(max(segs[0].first, 0.0), 1.0),
now, sum = 0;
int cnt = segs[0].second;
for(int j = 1; j < segs.size(); ++j) {
now = min(max(segs[j].first, 0.0), 1.0);
if (!cnt) sum += now - pre;
cnt += segs[j].second;
pre = now;
ans += cross(a, b) * sum;}}
return ans * 0.5;
}
// contains all points p such that: cross(b - a, p -
a) >= 0

```

6.13 Re-order Polygon

```

// rotate the polygon such that the (bottom, left)-most
point is at the first position
void reorder_polygon(vector<pt> &p) {
int pos = 0;
for (int i = 1; i < p.size(); i++) {
if (p[i].y < p[pos].y || (sign(p[i].y -
p[pos].y) == 0 && p[i].x < p[pos].x)) pos
= i;
}
rotate(p.begin(), p.begin() + pos, p.end());
}

```

6.14 Rectangle Union

```

const int N = 1e5 + 9;
struct Line {
int x, ymin, ymax, ty ;
Line() {}
Line(int _x, int _ym, int _ymx, int _ty) { x = _x
; ymin = _ym ; ymax = _ymx ; ty = _ty ; }
bool operator < (const Line &e) const { return x <
e.x ; }
};
Line all[N * 2];
long long t[N * 16], mp[N * 2], lazy[N * 16];
inline void push(int n, int b, int e) {

```

```

if (!lazy[n]) return;
t[n] = mp[e] - mp[b] - t[n];
if (b != e) {
lazy[n <= 1] ^= lazy[n];
lazy[n <= 1 | 1] ^= lazy[n];
}
lazy[n] = 0;
void upd(int n, int b, int e, int i, int j, int ty) {
push(n, b, e);
if (mp[e] < i || mp[b] > j) return;
if (mp[b] >= i && mp[e] <= j) {
lazy[n] ^= ty;
push(n, b, e);
return;
}
if (b + 1 >= e) return;
int mid = b + e >> 1, l = n <= 1, r = l | 1;
upd(l, b, mid, i, j, ty);
upd(r, mid, e, i, j, ty);
t[n] = t[l] + t[r];
}
int main() {
int n; cin >> n;
int id = 0, m = 0;
for (int i = 0; i < n; i++) {
int x1, y1, x2, y2; scanf("%d %d %d %d", &x1,
&y1, &x2, &y2);
assert(x1 < x2); assert(y1 < y2);
all[id++] = Line(x1, y1, y2, 1);
all[id++] = Line(x2, y1, y2, 1);
mp[++m] = y1 ; mp[++m] = y2 ;
}
n = id;
sort(all, all + n);
sort(mp + 1, mp + m + 1);
m = unique(mp + 1, mp + m + 1) - mp - 1;
upd(1, 1, m, all[0].ymin, all[0].ymax, all[0].ty);
long long ans = 0 ; // sum of area covered by odd
number of rectangles
for (int i = 1; i < n; i++) {
ans += 1LL * t[i] * (all[i].x - all[i - 1].x);
upd(1, 1, m, all[i].ymin, all[i].ymax,
all[i].ty);
}
cout << ans << '\n';
return 0;
}
// https://vjudge.net/problem/Gym-101982F

```

6.15 Rotating Calipers

```

// cross(o, a, b) = (a - o) X (b - o)
ll cross(const point &o, const point &a, const point
&b) {
return (a.x - o.x) * (b.y - o.y)
- (a.y - o.y) * (b.x - o.x);
}
// Rotating calipers to find max squared distance on
convex polygon H
ll max_pair_dist2(const vector<point> &H) {
int m = H.size();
if (m < 2) return 0;
if (m == 2) return DIST2(H[0], H[1]);
ll best = 0; int j = 1;
for (int i = 0; i < m; ++i) {
while (abs(cross(H[i], H[(i+1)%m], H[(j+1)%m]))
> abs(cross(H[i], H[(i+1)%m], H[j]))) {
j = (j+1) % m;
}
best = max(best, (ll)DIST2(H[i], H[j]));
best = max(best, (ll)DIST2(H[(i+1)%m], H[j]));
}
return best;
}

```

6.16 Ternary Search

```

double ternary_search(double l, double r) {
double eps = 1e-9;
while (r - l > eps) {

```

```

double m1 = l + (r - l) / 3;
double m2 = r - (r - l) / 3;
double f1 = f(m1);
double f2 = f(m2);
if (f1 < f2) l = m1;
else r = m2;
}
return f(l);
}

```

7 Graph

7.1 Articulation_Bridge

```

// br = bridges, p = parent
vector<pair<int, int>> br;
int dfsBR(int u, int p) {
low[u] = disc[u] = ++Time;
for (int& v : adj[u]) {
if (v == p) continue; // we don't want to go
back through the same path.
// if we go back is because we found another
way back
if (!disc[v]) { // if v has not been discovered
before
dfsBR(v, u); // recursive DFS call
if (disc[u] < low[v]) // condition to find
a bridge
br.push_back({u, v});
low[u] = min(low[u], low[v]); // low[v]
might be an ancestor of u
} else // if v was already discovered means
that we found an ancestor
low[u] = min(low[u], disc[v]); // finds the
ancestor with the least discovery
time
}
}
void BR() {
low = disc = vector<int>(adj.size());
Time = 0;
for (int u = 0; u < adj.size(); u++)
if (!disc[u])
dfsBR(u, u);
}

```

7.2 Articulation_Point

```

// adj[u] = adjacent nodes of u
// ap = AP = articulation points
// p = parent
// disc[u] = discovery time of u
// low[u] = 'low' node of u
int dfsAP(int u, int p) {
int children = 0;
low[u] = disc[u] = ++Time;
for (int& v : adj[u]) {
if (v == p) continue; // we don't want to go
back through the same path.
// if we go back is because we found another
way back
if (!disc[v]) { // if v has not been discovered
before
children++;
dfsAP(v, u); // recursive DFS call
if (disc[u] <= low[v]) // condition #1
ap[u] = 1;
low[u] = min(low[u], low[v]); // low[v]
might be an ancestor of u
} else // if v was already discovered means
that we found an ancestor
low[u] = min(low[u], disc[v]); // finds the
ancestor with the least discovery time
}
}

```

```

return children;
void AP() {
    ap = low = disc = vector<int>(adj.size());
    Time = 0;
    for (int u = 0; u < adj.size(); u++)
        if (!disc[u])
            ap[u] = dfsAP(u, u) > 1;
}

```

7.3 bellmanford

```

struct Edge{
    ll from, to, cost;};
ll vertex, edges; // no. of vertex and edges in
// the graph
vector<Edge> vec(200005); // store the edges from,to
// and cost.
vector<ll> dist(200005, INF);
vector<ll> parent(200005, -1);
void Bellman_Ford(){
    dist[1] = 0;
    for (ll i = 1; i < vertex; i++){
        for (Edge it : vec){
            if (dist[it.from] < INF){
                if (dist[it.from] + it.cost < dist[it.to]){
                    dist[it.to] = max(dist[it.from] + it.cost,
                        -INF);
                    parent[it.to] = it.from;}}}}
void retrieve_shortest_path(vector<ll> &path){
    if (dist[vertex] == INF)
        return; // NO Path Exist
    for (ll current = vertex; current != -1; current =
        parent[current]){
        path.push_back(current);
    }
    reverse(path.begin(), path.end());
}
void find_negative_cycle(vector<ll> &path){
    // run this after bellman ford has been run
    ll start = -1;
    for (Edge it : vec){
        if (dist[it.from] < INF){
            if (dist[it.from] + it.cost < dist[it.to]){
                dist[it.to] = dist[it.from] + it.cost;
                start = it.to;
                parent[it.to] = it.from;}}
    }
    if (start == -1)
        return; // no Negative cycle found so, path
        // is empty
    for (ll times = 0; times < vertex; times++){
        start = parent[start];
        path.push_back(start);
        for (ll current=parent[start];;current=parent[c]
            current){
            path.push_back(current);
            if (current==start)break;
        }
        reverse(path.begin(), path.end());
    }
}

```

7.4 BFS for All

```

const ll MAXN=2e5+5;
vector<ll> graph[MAXN];
vector<ll> level(MAXN, INF);
vector<ll> par(MAXN, -1);
vector<ll> height(MAXN, 0);
vector<ll> subTreeSize(MAXN, 0);
vector<bool> vis(MAXN, false);
void bfs(ll root) {
    queue<ll> q; q.push(root);
    par[root]=-1; level[root]=0;
    vis[root]=true; vector<ll> order;
    while(!q.empty()) {
        ll node=q.front(); q.pop();
    }
}

```

```

order.push_back(node);
subTreeSize[node]=1;
for(auto child:graph[node]) {
    if(!vis[child]) {
        vis[child]=true; par[child]=node;
        level[child]=level[node] + 1;
        q.push(child);}}
reverse(order.begin(), order.end());
for(ll node:order) {
    if(par[node]!=-1){
        height[par[node]] = max(height[par[node]],
            height[node] + 1);
        subTreeSize[par[node]] +=
            subTreeSize[node];}}
}

```

7.5 BiconnectedComponents

```

// Description: Finds all biconnected components in an
// undirected graph, and
// runs a callback for the edges in each. In a
// biconnected component there
// are at least two distinct paths between any two
// nodes. Note that a node can
// be in several components. An edge which is not in
// a component is a bridge,
// i.e., not part of any cycle.
// Usage:
// int eid = 0; ed.resize(N);
// for each edge (a,b) {
//     ed[a].emplace_back(b, eid);
//     ed[b].emplace_back(a, eid++);
// }
// bicomps(f)(const vi& edgelist) {...};
// Time: O(E + V)
#pragma once
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
    int me = num[at] = ++Time, top = me;
    for (auto [y, e] : ed[at]) if (e != par) {
        if (num[y]) {
            top = min(top, num[y]);
            if (num[y] < me)
                st.push_back(e);
        } else {
            int si = sz(st);
            int up = dfs(y, e, f);
            top = min(top, up);
            if (up == me) {
                st.push_back(e);
                f(vi(st.begin() + si, st.end()));
                st.resize(si);
            } else if (up < me) st.push_back(e);
            else { /* e is a bridge */ }
        }
    }
    return top;
}
template<class F>
void bicomps(F f) {
    num.assign(sz(ed), 0);
    rep(1, 0, sz(ed)) if (!num[i]) dfs(i, -1, f);
}

```

7.6 Binary Lifting - Lowest Common Ancestor (LCA)

```

const ll MAXN=2e5+5;
const ll LOG=log2(1e6);
vector<ll> par(MAXN, -1);
vector<ll> depth(MAXN, 0);
vector<vector<ll>> anc(MAXN, vector<ll>(LOG, 1));
ll kthanc(ll node, ll k){
    if(depth[node]<k){return -1;}
}

```

```

loop(i, 0, LOG){
    if(k&(1<<i)){node=anc[node][i];}
    return node;}
void Buildanc(ll root, ll end_node){
    par[root]=root;
    loop(i, 0, LOG){
        loop(node, root, end_node+1){
            if(i==0){
                anc[node][i]=par[node]; continue;
            }
            if(node!=root)
                anc[node][i]=anc[anc[node][i-1]][i-1];}}
}
ll LCA(ll a, ll b){
    if(depth[a]<depth[b]){swap(a,b);}
    ll k=depth[a]-depth[b];
    a=kthanc(a, k);
    if(a==b) return a;
    for(ll i=LOG-1; i>=0; i--){
        if(anc[a][i]!=anc[b][i]){
            a=anc[a][i]; b=anc[b][i];}
    }
    return anc[a][0];
}

```

7.7 CentroidDecomposition

```

const int N = 200005;
ll ans = 0;
int K1, K2; // path length k that we r searching
struct CentroidDecomposition{
    vector<vector<int>> adj;
    vector<bool> done;
    vector<int> parent, subsize, cnt, totalcnt;
    int root, max depth = 1, subtreedepth = 0;
    CentroidDecomposition(vector<vector<int>> &tree) :
        adj(tree){
            int siz = adj.size();
            done.assign(siz, false);
            parent.assign(siz, -1);
            subsize.assign(siz, 0);
            cnt.assign(N, 0);
            totalcnt.assign(N, 0);
            totalcnt[0] = 1;
            decompose(0);
        }
    int get_subtree_size(int node, int parent = -1){
        subsize[node] = 1;
        for (auto it : adj[node]){
            if (it == parent || done[it])
                continue;
            subsize[node] += get_subtree_size(it, node);
        }
        return subsize[node];
    }
    int get_centroid(int node, int need, int parent = -1){
        ll ret = node;
        for (auto it : adj[node]){
            if (it != parent && !done[it] && subsize[it]
                >= need)
                return get_centroid(it, need, node);
        }
        return ret;
    }
    void get_cnt(int node, int parent, bool filling, int
        depth = 1){
        if (depth > K2)
            return;
        subtreedepth = max(subtreedepth, depth);
        if (filling){
            cnt[depth]++;
        }
        for (auto it : adj[node])
            if (!done[it] && it != parent)
                get_cnt(it, node, filling, depth + 1);
    }
    void decompose(int node = 0){
        int centroid = get_centroid(node,
            get_subtree_size(node) >> 1LL);
    }
}

```

```

done[centroid] = 1;
max_depth = 0;
ll partialsum = 0, init = (K1==1);
// cout<<centroid+1<<endl;
for (auto it : adj[centroid]){
    if (!done[it]){
        subtreedepth = 0;
        get_cnt(it, centroid, true);
        max_depth = max(max_depth, subtreedepth);
        partialsum=init;
        for (ll i=1;i<=subtreedepth;i++){
            ans+=partialsum*cnt[i];
            if (i<K1)partialsum+=totalcnt[K1-i-1];
            if (i<K2)partialsum-=totalcnt[K2-i];}
        for (ll i=1;i<=subtreedepth;i++){
            totalcnt[i]+=cnt[i];}
        for (ll i=K1-1;i<=K2-1 &&
            i<=subtreedepth;i++){init+=cnt[i];
            fill(cnt.begin(), cnt.begin()+subtreedepth+1, 0);}}
fill(totalcnt.begin() + 1, totalcnt.begin() +
    max_depth + 1, 0);
for (auto it : adj[centroid])
    if (!done[it])
        decompose(it);};

```

7.8 DFS Plus for tree

```

vector<ll> graph[N];
vector<ll> euler(N,-1);
vector<ll> subtree_size(N,-1);
vector<ll> position_in_euler(N,-1);
vector<ll> heavy_child(N,-1);
vector<ll> depth(N,0);
vector<ll> height(N,0);
vector<ll> par(N,0);
ll idx=0;
void dfs_plus(ll node, ll parent){
    par[node]=parent;
    // for subtree range:
    // for(int
    i=pos[node];i<pos[node]+subtree_size[node];i++)
        node=euler[i]
    position_in_euler[node]=idx;
    euler[idx++]=node;
    subtree_size[node]=1;
    heavy_child[node]=-1; // leaf nodes remain -1
    height[node]=0;
    for(auto child:graph[node]){
        if(child==parent) continue;
        depth[child]=depth[node]+1;
        dfs_plus(child,node);
        height[node]=max(height[node],height[child]+1);
        subtree_size[node]+=subtree_size[child];
        if(heavy_child[node]==-1 || subtree_size[heavy_
            child[node]]<subtree_size[child]){
            heavy_child[node]=child;}}
}

```

7.9 Dinic

```

struct edge{
    int to, from;
    ll capacity, flow;
    edge() {}
    edge(int from, int to, ll capacity) : to(to),
        from(from), capacity(capacity), flow(0) {};}
struct Dinic{
    int num_nodes;
    vector<vector<int>> adj;
    vector<edge> edges;
    vector<int> level;

```

```

    vector<int> ptr;
    Dinic(int n) : num_nodes(n), adj(n), level(n), ptr(n)
        {}
    void add_edge(int from, int to, ll capacity){
        if (from ^ to){
            adj[from].push_back(edges.size());
            edges.push_back(edge(from, to, capacity));
            adj[to].push_back(edges.size());
            edges.push_back(edge(to, from, 0)); // Residual
            edge;}
    bool bfs(int s, int t){
        level.assign(num_nodes, num_nodes + 1);
        level[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty()){
            int node = q.front();
            q.pop();
            if (node == t)
                break;
            for (int id : adj[node]){
                auto &e = edges[id];
                if (e.capacity > e.flow && level[e.to] >
                    level[e.from] + 1){
                    level[e.to] = level[e.from] + 1;
                    q.push(e.to);}}
        return level[t] != num_nodes + 1;}
    ll dfs(int node, int t, ll flow = -1){
        if (node == t || flow == 0)
            return flow;
        for (int &i = ptr[node]; i < adj[node].size();
            i++){
            edge &e = edges[adj[node][i]];
            edge &oe = edges[adj[node][i] ^ 1];
            if (level[e.to] == level[e.from] + 1){
                ll amt = e.capacity - e.flow;
                if (flow != -1 && amt > flow)
                    amt = flow;
                if (ll pushed = dfs(e.to, t, amt)){
                    e.flow += pushed;
                    oe.flow -= pushed;
                    return pushed;}}
            return 0LL;}
    ll maxflow(int s, int t){
        ll total = 0;
        while (bfs(s, t)){
            fill(ptr.begin(), ptr.end(), 0);
            while (ll flow = dfs(s, t))
                total += flow;}
        return total;}
    vector<pair<int, int>> min_cut(int s, int t) {
        maxflow(s, t);
        vector<bool> visited(num_nodes);
        queue<int> q;
        q.push(s);
        visited[s] = true;
        while (!q.empty()) {
            int node = q.front();
            q.pop();
            for (int id : adj[node]) {
                if (edges[id].capacity - edges[id].flow >
                    0 && !visited[edges[id].to]) {
                    visited[edges[id].to] = true;
                    q.push(edges[id].to);}}
        vector<pair<int, int>> cut_edges;
        for (const auto &e : edges) {
            if (visited[e.from] && !visited[e.to] &&
                e.capacity > 0) {
                cut_edges.push_back({e.from, e.to});}
        return cut_edges;};

```

7.10 DPOnTree

```

void dfs2(int node, int par, vvi &g){
    vll childs;
    for (auto ch: g[node]){
        if (ch == par) continue;
        childs.pb(ch);}
    ll sz = childs.size();
    if (sz == 0) return;
    vll val(sz), suf(sz), pref(sz);
    FOR(i, 0, sz) val[i] = downDP[childs[i]] + 1;
    suf[sz - 1] = val[sz - 1]; pref[0] = val[0];
    FOR(i, 1, sz) pref[i] = max(val[i], pref[i - 1]);
    FORR(i, sz - 2, 0) suf[i] = max(val[i], suf[i +
        1]);
    FOR(i, 0, sz){
        ll mx = upDp[node];
        i == 0? : mx = max(mx, pref[i - 1]);
        i == sz - 1? : mx = max(mx, suf[i + 1]);
        upDp[childs[i]] = max(upDp[childs[i]], 1 + mx);
        dfs2(childs[i], node, g);}
}

```

7.11 DSU

```

vector<ll> par(N,-1);
vector<ll> set_size(N,0LL);
// initialize the DSU with make_set() on all nodes
void make_set(ll node){
    par[node]=node;
    set_size[node]=1;}
ll find_set(ll node){
    if(node==par[node]){return node;}
    return par[node]=find_set(par[node]);}
void union_sets(ll a, ll b){
    a=find_set(a);b=find_set(b);
    if(a!=b){
        if(set_size[a]<set_size[b]) swap(a,b);
        par[b]=a;
        set_size[a]+=set_size[b];}
}

```

7.12 DsuOnTree

```

void dfs(int u, int p) {
    node[tt] = u;
    tin[u] = tt++, sz[u] = 1, hc[u] = -1;
    for (auto v : adj[u]) {
        if (v != p) {
            dfs(v, u);
            sz[u] += sz[v];
            if (hc[u] == -1 or sz[hc[u]] < sz[v]) {
                hc[u] = v;}}
    tout[u] = tt - 1;}
void dsu(int u, int p, int keep) {
    for (int v : adj[u]) {
        if (v != p and v != hc[u]) {
            dsu(v, u, 0);}}
    if (hc[u] != -1) {
        dsu(hc[u], u, 1);}
    for (auto v : adj[u]) {
        if (v != p and v != hc[u]) {
            for (int i = tin[v]; i <= tout[v]; ++i) {
                int w = node[i];
                // get ans in case of ans is related to
                simple path or pair
            }
            for (int i = tin[v]; i <= tout[v]; ++i) {
                int w = node[i];
                // Add contribution of node w
            }
        }
    }
}

```

```

    }
    // Add contribution of node u
    // get ans in case ans is related to subtree
    if (!keep) {
        for (int i = tin[u]; i <= tout[u]; ++i) {
            int w = node[i];
            // remove contribution of node w
            // Data structure in initial state (empty
            // contribution)
        }
    }
    dfs(0, 0);
    dsu(0, 0, 0);

```

7.13 eulerpathdirected

```

const int N = 200005;
ll num, edges;
vector<vector<ll>> adj(N);
vector<ll> circuit;
vector<bool> visited(N);
vector<ll> from(N), to(N), outdegree(N), indegree(N);
void eulerpathdirected(ll start){
    vector<ll> path;
    path.push_back(start);
    ll current = start;
    while (!path.empty()){
        if (outdegree[current]){
            path.push_back(current);
            while (visited[adj[current].back()]){
                adj[current].pop_back();
            }
            ll edge = adj[current].back(), next = to[edge];
            visited[edge] = 1;
            outdegree[current]--, indegree[next]--;
            current = next;
        }
        else{
            circuit.push_back(current);
            current = path.back();
            path.pop_back();
        }
    }
    reverse(circuit.begin(), circuit.end());
}
void solve(){
    cin >> num >> edges;
    for (ll i = 0; i < edges; i++){
        ll u, v;
        cin >> u >> v;
        from[i] = --u;
        to[i] = --v;
        adj[u].push_back(i);
        // adj[v].push_back(i);
        outdegree[u]++;
        indegree[v]++;
    }
    bool pos = (outdegree[0] == indegree[0] + 1 &&
        ~ indegree[num - 1] == outdegree[num - 1] + 1);
    for (ll i = 1; i < num - 1; i++){
        if (indegree[i] != outdegree[i]){
            pos = 0;
        }
    }
    eulerpathdirected(0);
    if (!pos || circuit.size() != edges + 1){
        cout << "IMPOSSIBLE";
        return;
    }
    for (auto it : circuit)
        cout << it + 1 << " ";
    cout << endl;
}

```

7.14 eulerpathundirected

```

const int N = 200005;
ll num, edges;
vector<vector<ll>> adj(N);
vector<ll> circuit;
vector<bool> visited(N);
vector<ll> from(N), to(N), degree(N);

```

```

void eulerpathundirected(ll start){
    vector<ll> path;
    path.push_back(start);
    ll current = start;
    while (!path.empty()){
        if (degree[current]){
            path.push_back(current);
            while (visited[adj[current].back()]){
                adj[current].pop_back();
            }
            ll edge = adj[current].back(), next = current
                ~ ^ from[edge] ^ to[edge];
            visited[edge] = 1;
            degree[current]--, degree[next]--;
            current = next;
        }
        else{
            circuit.push_back(current);
            current = path.back();
            path.pop_back();
        }
    }
    reverse(circuit.begin(), circuit.end());
}
void solve(){
    cin >> num >> edges;
    for (ll i = 0; i < edges; i++){
        ll u, v;
        cin >> u >> v;
        from[i] = --u;
        to[i] = --v;
        adj[u].push_back(i);
        adj[v].push_back(i);
        degree[u]++;
        degree[v]++;
    }
    bool pos = 1, wrong = 0;
    for (ll i = 0; i < num; i++){
        if (degree[i] & 1){
            if (wrong)
                pos = 0;
            else
                wrong = 1;
        }
    }
    eulerpathundirected(0);
    if (!pos || circuit.size() != edges + 1){
        cout << "IMPOSSIBLE";
        return;
    }
    for (auto it : circuit)
        cout << it + 1 << " ";
    cout << endl;
}

```

7.15 FloydWarshall

```

const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m) {
    int n = sz(m);
    rep(i,0,n) m[i][i] = min(m[i][i], 0LL);
    rep(k,0,n) rep(i,0,n) rep(j,0,n)
        if (m[i][k] != inf && m[k][j] != inf) {
            auto newDist = max(m[i][k] + m[k][j],
                ~ -inf);
            m[i][j] = min(m[i][j], newDist);
        }
    rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,n)
        if (m[i][k] != inf && m[k][j] != inf) m[i][j]
            ~ = -inf;
}

```

7.16 gridBFS+DFS

```

bool vis[1005][1005];
ll level[1005][1005];
vector<pair<ll, ll>> nodelist;
vector<ll> dx={0,0,1,-1}, dy={-1,1,0,0};
vector<vector<char>>
    ~ grid(1005, vector<char>(1005, '#'));
void dfs(ll i, ll j, ll n, ll m){

```

```

    if(i<0 || i>=n || j<0 || j>=m) return;
    if(grid[i][j]!='.') return;
    if(vis[i][j]) return;
    vis[i][j]=true;
    loop(k,0,4){
        dfs(dx[k]+i,dy[k]+j,n,m);
    }
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    pll source_node, dest_node;
    ll n,m;cin>>n>>m;
    loop(i,0,n){
        loop(j,0,m){
            cin>>grid[i][j];
            //if(grid[i][j]!='.') nodelist.pb({i,j});
            if(grid[i][j]=='A') source_node={i,j};
            if(grid[i][j]=='B') dest_node={i,j};
        }
    }
    queue<pll> q;
    q.push(source_node);
    while(!q.empty()){
        pll curr_node=q.front();
        q.pop();
        loop(k,0,4){
            ll x=dx[k]+curr_node.ft;
            ll y=dy[k]+curr_node.sc;
            if(x>=0 && x<n && y>=0 && y<m){
                if(!vis[x][y] && (grid[x][y]!='.' ||
                    ~ grid[x][y]!='B')){
                    q.push({x,y});vis[x][y]=1;
                    level[x][y]=level[curr_node.ft][curr_node.sc]+
                        ~ 1;
                }
            }
        }
        ll totallev=level[dest_node.ft][dest_node.sc];
        if(totallev==0){No;return 0;}
        Yes;cout<<totallev<<endl;
    }
}

```

7.17 HLD2

```

// Description: Heavy Light Decomposition for trees.
// 1-indexed nodes.
SegTree t;
vector<int> g[MAX];
int par[MAX][LOGN+1], depth[MAX], sz[MAX];
int T=0, head[MAX], st[MAX], en[MAX];
void dfs(int node, int p=0){
    par[node][0]=p;
    sz[node]=1;
    if(p!=0)
        g[node].erase(find(g[node].begin(), g[node].end(),
            ~ (p)));
    for(auto &ch:g[node]){
        if(ch==p) continue;
        depth[ch]=depth[node]+1;
        dfs(ch, node);
        sz[node]+=sz[ch];
        if(sz[ch]>sz[g[node][0]])
            swap(ch, g[node][0]);
    }
}
void dfs_hld(int node){
    st[node]=++T;
    for(auto ch:g[node]){
        if(ch==g[node][0]) head[ch]=head[node];
        else head[ch]=ch;
        dfs_hld(ch);
    }
    en[node]=T;
}
int query_up(int u, int v){
    ll ans=-INF;
    while(head[u]!=head[v]){
        ans=max(ans, t.getRes(st[head[u]], st[u], 0, 0, t.s
            ~ z-1));
        u=par[head[u]][0];
    }
    return ans;
}

```

```

ans=max(ans,t.getRes(st[v],st[u],0,0,t.sz-1));
return ans;}
int query(int u,int v){
int lca=LCA(u,v);
int ans=query_up(u,lca);
if(v!=lca)
ans=max(ans,query_up(v,kthAncestor(v,depth[v]-
depth[lca]-1)));
return ans;}

```

7.18 MST Prim's

```

void solve(){
ll n,m;
cin>>n>>m;
priority_queue<pair<ll,ll>, vector<pair<ll,ll>>,
greater<pair<ll,ll>>> pq;
loop(i,0,m){
ll u,v,wt;cin>>u>>v>>wt;
graph[u].push_back({v,wt});
graph[v].push_back({u,wt});}
ll totalcost=0;ll count=0;
pq.push({0,1});
while(!pq.empty() && count<n){
auto [cost,node]=pq.top();
pq.pop();
if(vis[node]) continue;
vis[node]=true;totalcost+=cost;
count++;
for(auto [child,wt]:graph[node]){
if(vis[child]) continue;
pq.push({wt,child});}}
if(count==n) cout<<totalcost<<endl;
else cout<<"IMPOSSIBLE\n";}

```

7.19 SCC2

```

// Description: 1-indexed nodes.
ll num, edges;
vector<vector<ll>> adj, reverseadj;
vector<ll>visited, visitororder, ans;
vector<vector<ll>>allsc;
void dfs(ll node){
visited[node]=1;
for(auto it:adj[node]){
if(!visited[it])dfs(it);}
visitororder.push_back(node);}
void dfs2(ll node,ll times){
visited[node]=1;
ans[node]=times;
allsc[times-1].push_back(node);
for(auto it:reverseadj[node]){
if(!visited[it])
dfs2(it,times);}}
void kosaraju(){
for(ll i=0;i<num;i++){
if(!visited[i])dfs(i);}
reverse(visitororder.begin(),visitororder.end());
visited.assign(num,0);
ll scc=0;
for(auto i:visitororder){
if(!visited[i]){
dfs2(i,++scc);}}
cout<<scc<<endl;
for(auto it:ans)cout<<it<<" ";
for(ll i=0;i<scc;i++){
cout<<i<<" ";
for(auto it:allsc[i])cout<<it<<"
";cout<<endl;}}

```

7.20 Subtree queries - Number of nodes with color k in the subtree

```

const ll INF=1e16;
const double EPS = 1e-9;
const ll M=1e9+7;
const ll N=1e5+5;
set<ll> collec[N];
vector<ll> ans(N,-1);
vector<ll> graph[N];
vector<ll> euler(N,-1);
vector<ll> subtree_size(N,-1);
vector<ll> position_in_euler(N,-1);
vector<ll> heavy_child(N,-1);
ll idx=0;
vector<map<ll,ll>> answers(N);
vector<pll> qs;
vector<ll> color(N,-1);
vll cnt(N);
void dfs(ll node,ll par,bool keep){
for(auto child:graph[node]){
if(child!=par && child!=heavy_child[node]){
dfs(child,node,false);}}
if(heavy_child[node]!=-1)
dfs(heavy_child[node],node,true);
cnt[color[node]]++;
for(auto child:graph[node]){
if(child!=par && child!=heavy_child[node]){
for(int i=position_in_euler[child];i<position_in_euler[child]+subtree_size[child];i++){
ll vertex=euler[i];
cnt[color[vertex]]++;}}}
for(auto [a,b]:answers[node]){
b=cnt[a];}
if(!keep){
for(int i=position_in_euler[node];i<position_in_euler[node]+subtree_size[node];i++){
ll vertex=euler[i];
cnt[color[vertex]]--;}}}
void solve(){
ll n,q;cin>>n>>q;
vector<ll> distinct_colors;
for(ll i=1;i<=n;i++){
cin>>color[i];
distinct_colors.push_back(color[i]);}
sort(all(distinct_colors));
distinct_colors.erase(unique(all(distinct_colors)),
distinct_colors.end());
for(ll i=1;i<=n;i++){
color[i]=lower_bound(all(distinct_colors),color[i],r[i])-distinct_colors.begin()+1;}
loop(i,0,n-1){
ll a,b;
cin>>a>>b;
graph[a].push_back(b);
graph[b].push_back(a);}
while(q--){
ll a,b;
cin>>a>>b;
ll realb=lower_bound(all(distinct_colors),b)-distinct_colors.begin()+1;
answers[a][realb]=-1;
qs.push_back({a,realb});}
dfs_plus(1,-1);
dfs(1,-1,false);
for(auto [a,b]:qs){
cout<<answers[a][b]<<endl;}}
int main(){
ios base::sync with_stdio(0);
cin.tie(0); cout.tie(0);

```

solve();}

7.21 TopoSort

```

vi topoSort(const vector<vi>& gr) {
vi indeg(sz(gr)), q;
for (auto& li : gr) for (int x : li) indeg[x]++;
rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
rep(j,0,sz(q)) for (int x : gr[q[j]])
if (--indeg[x] == 0) q.push_back(x);
return q;}

```

7.22 TreeDiameter

```

void dfs(int u, int p){
int mx1 = 0, mx2 = 0;
for (int v : adj[u]) {
if (v == p) continue;
dfs(v, u); int d = f[v] + 1;
if (d > mx1) swap(d, mx1);
if (d > mx2) swap(d, mx2);}
f[u] = mx1;
diameter = max(diameter, mx1 + mx2);}

```

7.23 UnionFindRollback

```

#pragma once
struct DSU Rollback {
vector<ll> par, rnk, sz;
ll c;
struct State {
ll u, v, rnk_u, rnk_v, sz_v, prev_c;
};
vector<State> history; // history: stores changes
for rollback
DSU_Rollback(ll n) : par(n + 1), rnk(n + 1, 0),
sz(n + 1, 1), c(n) {
for (ll i = 1; i <= n; i++) par[i] = i;}
ll find(ll i) {
while (par[i] != i) i = par[i];
return i;}
bool same(ll i, ll j) { return find(i) == find(j);}
ll get_size(ll i) {return sz[find(i)];}
ll count() {return c;} // connected components
void merge(ll i, ll j) {
i = find(i); j = find(j);
if (i == j) {
history.push_back({-1, -1, -1, -1, -1, -1, c}); // store previous state
return;}
if (rnk[i] > rnk[j]) swap(i, j);
history.push_back({i, j, rnk[i], rnk[j],
sz[j], c});
par[i] = j;
sz[j] += sz[i];
c--;
if (rnk[i] == rnk[j]) rnk[j]++;}
void rollback() { //does one rollback
assert(!history.empty());
auto s = history.back();
history.pop_back();
c = s.prev_c;
if (s.u == -1) return; // nothing merged
ll i = s.u, j = s.v;
par[i] = i;
rnk[i] = s.rnk_u;
rnk[j] = s.rnk_v;
sz[j] = s.sz_v;
int snapshot() {

```

```

return history.size();}
void rollback_to(int snap) { //rollback to this
    snap
    while ((int)history.size() > snap){
        rollback();}
};

```

8 Math

8.1 Binary Operations

```

ll number_of_set_bits(ll n){
    return __builtin_popcountll(n);}
inline ll msb_idx(ll n) {
    if(n==0) return -1;
    return 63- __builtin_clzll(n); }
inline ll lsb_idx(ll n){
    if(n==0) return -1;
    return __builtin_ctzll(n);}
inline bool is_power_of_two(ll n) {
    return (n>0)&&((n&(n-1))==0);}
vector<ll> decimal_to_binary(ll n){
    vll ans;ll idx=0;
    while(n>0){
        ans.pb(n%2);n>>=1;}
    while(ans.size()<32) ans.pb(0);
    reverse(ans.begin(),ans.end());
    return ans;}

```

8.2 FFT

```

const ll N = 3e5 + 9;
const double PI = acos(-1);
struct base{
    double a, b;
    base(double a = 0, double b = 0) : a(a), b(b) {}
    const base operator+(const base &c) const{
        return base(a + c.a, b + c.b);}
    const base operator-(const base &c) const{
        return base(a - c.a, b - c.b);}
    const base operator*(const base &c) const{
        return base(a * c.a - b * c.b, a * c.b + b *
            c.a);}
};
void fft(vector<base> &p, bool inv = 0){
    ll n = p.size(), i = 0;
    for (ll j = 1; j < n - 1; ++j){
        for (ll k = n >> 1; k > (i ^ k); k >>= 1);
        if (j < i) swap(p[i], p[j]);
        for (ll l = 1; m = (l << 1) <= n; l <<= 1){
            double ang = 2 * PI / m;
            base wn =
                base(cos(ang), (inv?1:-1)*sin(ang)), w;
            for (ll i = 0, j, k; i < n; i += m){
                for (w = base(1,0), j=i, k=i+l; j<k; ++j, w=w*wn){
                    base t = w * p[j + l];
                    p[j + l] = p[j] - t;
                    p[j] = p[j] + t;}}
            if (inv
                for (ll i = 0; i < n; ++i)
                    p[i].a /= n, p[i].b /= n;}
    vector<long long> multiply(vector<ll> &a, vector<ll>
        &b){
        ll n = a.size(), m = b.size(), t = n + m - 1, sz =
            1;
        while (sz < t) sz <<= 1;
        vector<base> x(sz), y(sz), z(sz);
        for (ll i = 0; i < sz; ++i){
            x[i] = i<(ll)a.size()?base(a[i],0):base(0,0);
            y[i] = i<(ll)b.size()?base(b[i],0):base(0,0);}
        fft(x), fft(y);
        for (ll i = 0; i < sz; ++i) z[i] = x[i] * y[i];
    }
}

```

```

fft(z, 1);
vector<long long> ret(sz);
for (ll i = 0; i < sz; ++i) ret[i] = (long
    long) round(z[i].a);
// size of the resultant polynomial is two's power
return ret;}

```

8.3 Gaussian Elimination

```

// formation of the equation vector:
// [ 1 2 1 | 6 ] -> Represents x + 2y + z = 6
// [ 2 3 1 | 9 ] -> Represents 2x + 3y + z = 9
// [ 3 1 2 | 8 ] -> Represents 3x + y + 2z = 8
// Time Complexity : O(min(n,m)*n*m)
const double EPS = 1e-9;
const int INF = 2;
int gauss (vector < vector<double> > a, vector<double>
    & ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;
    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs
                (a[sel][col])) sel = i;
        if (abs (a[sel][col]) < EPS) continue;
        for (int i=col; i<m; ++i) swap (a[sel][i],
            a[row][i]);
        where[col] = row;
        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<m; ++j)
                    a[i][j] -= a[row][j] * c;
                ++row;
            }
        ans.assign (m, 0);
        for (int i=0; i<m; ++i)
            if (where[i] != -1)
                ans[i] = a[where[i]][m] / a[where[i]][i];
        for (int i=0; i<n; ++i) {
            double sum = 0;
            for (int j=0; j<m; ++j) sum += ans[j] * a[i][j];
            if (abs (sum - a[i][m]) > EPS) return 0;
        }
        for (int i=0; i<m; ++i)
            if (where[i] == -1) return INF;
        return 1;
    }
}

```

8.4 Matrix

```

const ll N = 2e5 + 1;
using Matrix = vector<vector<ll>>;
ll binpow(ll a, ll n, ll m){
    if(n==0) return 1ll;
    ll x=binpow(a,n/2,m);
    if(n&1ll) return (a*((x*x)%m))%m;
    else return (x*x)%m;}
ll modInverse(ll n, ll m){
    return binpow(n,m-2,m);}
//optional MOD
Matrix multiply(const Matrix &A, const Matrix &B, ll
    MOD = -1) {
    // A = n x p
    // B = p x m
    int n = A.size();
    int m = B[0].size();
    int p = B.size();
    Matrix C(n, vector<ll>(m, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {

```

```

            for (int k = 0; k < p; k++) {
                C[i][j] += A[i][k] * B[k][j];
                if (MOD != -1) C[i][j] %= MOD;}}
        return C;
    }
// Identity matrix of size n
Matrix identity(ll n) {
    Matrix I(n, vector<ll>(n, 0));
    for (ll i = 0; i < n; i++) I[i][i] = 1;
    return I;
}
// Fast exponentiation of matrix A^power with optional
    modulo
Matrix matrix_power(Matrix A, ll power, ll MOD=-1){
    ll n = A.size();
    Matrix result = identity(n);
    while(power>0) {
        if(power&1){
            result = multiply(result, A, MOD);
            A = multiply(A, A, MOD);
            power/=2;}
        return result;}
// Computes the determinant of matrix A under modulo
    MOD
ll determinant(Matrix A, ll MOD) {
    int n = A.size(); ll det = 1;
    for (int i = 0; i < n; i++) {
        int pivot = i;
        for (int j = i + 1; j < n; j++) {
            if (A[j][i] != 0) {
                pivot = j;
                break;}}
        if (pivot != i) {
            swap(A[i], A[pivot]);
            det = (MOD - det) % MOD; }
        if (A[i][i] == 0) return 0;
        det = (det * A[i][i]) % MOD;
        ll inv = modInverse(A[i][i], MOD);
        for (int j = i + 1; j < n; j++) {
            ll factor = (A[j][i] * inv) % MOD;
            for (int k = i; k < n; k++) {
                A[j][k] = (A[j][k] - factor * A[i][k])
                    % MOD;
                if (A[j][k] < 0) A[j][k] += MOD;}}
        return (det % MOD + MOD) % MOD; }
// Computes the inverse of matrix A under modulo MOD
// Returns an empty matrix if A is not invertible
    (singular)
Matrix inverse(Matrix A, ll MOD) {
    int n = A.size();
    Matrix inv = identity(n);
    for (int i = 0; i < n; i++) {
        int pivot = i;
        for (int j = i + 1; j < n; j++) {
            if (A[j][i] != 0) {
                pivot = j;
                break;}}
        if (A[pivot][i] == 0) return {};
        if (pivot != i) {
            swap(A[i], A[pivot]);
            swap(inv[i], inv[pivot]);}
        ll invPivot = modInverse(A[i][i], MOD);
        for (int j = 0; j < n; j++) {
            A[i][j] = (A[i][j] * invPivot) % MOD;
            inv[i][j] = (inv[i][j] * invPivot) % MOD; }
        for (int j = 0; j < n; j++) {
            if (i != j) {
                ll factor = A[j][i];
                for (int k = 0; k < n; k++) {
                    A[j][k] = (A[j][k] - factor * A[i][k]) %
                        MOD;
                    if (A[j][k] < 0) A[j][k] += MOD; }
            }
        }
    }
}

```

```

    inv[j][k] = (inv[j][k] - factor *
        inv[i][k]) % MOD;
    if (inv[j][k] < 0) inv[j][k] += MOD;}}}
return inv;

```

8.5 Mobius Precomputation

```

const ll N=3e6+5;
// call SIEVE() and PRIMES() in int main()
vector<bool> sieve(N,true);
vector<ll> primes;
vector<ll> mu(N,1);
void Build mu(){
    mu[1]=1;
    for(auto p:primes){
        ll sq=p*p;
        for(ll j=sq;j<N;j+=sq){
            mu[j]=0;
        }
        for(auto p:primes){
            for(ll j=p;j<N;j+=p)mu[j]*=-1;
        }
    }
}

```

8.6 ntt

```

const int mod = 998244353;
const int root = 15311432;
const int k = 1 << 23;
int root_1;
vector<int> rev;
void pre(int sz){
    root_1 = bigmod(root, mod - 2, mod);
    if (rev.size() == sz) return;
    rev.resize(sz);
    rev[0] = 0;
    int lg_n = __builtin_ctz(sz);
    for (int i = 1; i < sz; ++i)
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (lg_n - 1));
    void f(vector<int> &a, bool inv){
        int n = a.size();
        for (int i = 1; i < n - 1; ++i) if (i < rev[i])
            swap(a[i], a[rev[i]]);
        for (int len = 2; len <= n; len <= 1) {
            int wlen = inv ? root_1 : root;
            for (int i = len; i < k; i <= 1) wlen = 1ll *
                wlen * wlen % mod;
            for (int st = 0; st < n; st += len){
                int w = 1;
                for (int j = 0; j < len / 2; j++){
                    int ev = a[st + j];
                    int od = 1ll * a[st + j + len / 2] * w
                        % mod;
                    a[st + j] = ev + od < mod ? ev + od :
                        ev + od - mod;
                    a[st + j + len / 2] = ev - od >= 0 ?
                        ev - od : ev - od + mod;
                    w = 1ll * w * wlen % mod;
                }
            }
        }
        if (inv){
            int n1 = bigmod(n, mod - 2, mod);
            for (int i = 0; i < n; ++i) a[i] = a[i] * n1 % mod;
        }
    }
    vector<int> mul(vector<int> &a, vector<int> &b){
        int n = a.size(), m = b.size(), sz = 1;
        while (sz < n + m - 1) sz <= 1;
        vector<int> x(sz), y(sz), z(sz);
        for (int i = 0; i < sz; ++i){
            x[i] = i < n ? a[i] : 0;
            y[i] = i < m ? b[i] : 0;
        }
        pre(sz); fft(x, 0); fft(y, 0);
        for (int i = 0; i < sz; ++i) z[i] = 1ll * x[i] *
            y[i] % mod;
    }
}

```

```

fft(z, 1); z.resize(n + m - 1);
return z;

```

8.7 PolyRoots

```

// Description: Finds the real roots to a polynomial.
// Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve
// Time: O(n^2 \log(1/epsilon))
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        rep(i,1,sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for (int i=sz(a)-1; i--;) c = a[i], a[i] =
            a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
vector<double> polyRoots(Poly p, double xmin, double
    xmax) {
    if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
    vector<double> ret; Poly der=p.der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.push_back(xmin-1);
    dr.push_back(xmax+1);
    sort(all(dr));
    rep(i,0,sz(dr)-1) {
        double l = dr[i], h = dr[i+1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            rep(it,0,60) { // while (h - l > 1e-8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign) l = m;
                else h = m;
            }
            ret.push_back((l + h) / 2);
        }
    }
    return ret;
}

```

8.8 XorBasis

```

const ll B = 30;
struct XorBasis {
    ll basis[B], id[B], sz;
    void clear() {
        memset(basis, 0, sizeof(basis));
        memset(id, 0, sizeof(id));
        sz = 0;
    }
    XorBasis() {
        clear();
    }
    void insert(ll x, ll j) {
        for (ll i = B - 1; i >= 0; i--) {
            if ((x >> i) & 1) {
                if (basis[i]) {
                    if (j > id[i]) {
                        swap(basis[i], x);
                        swap(id[i], j);
                    }
                    x ^= basis[i];
                }
                else {
                    basis[i] = x;
                    id[i] = j;
                    sz++;
                    break;
                }
            }
        }
    }
    // is there any subset having xor=x
    bool can(ll x) {
        for (ll i = B - 1; i >= 0; i--) {
            x = min(x, x ^ basis[i]);
        }
        return x == 0;
    }
}

```

```

// max subset xor after xoring with x
ll max_xor(ll x = 0, ll l = 0) {
    for (ll i = B - 1; i >= 0; i--) {
        if (id[i] >= l)
            x = max(x, x ^ basis[i]);
    }
    return x;
}
// kth smallest xor, 1st is 0
ll kth(ll k) {
    if ((k < 1) || ((1 << sz) < k))
        return -1;
    ll x = 0;
    ll cnt = (1 << sz);
    for (ll i = B - 1; i >= 0; i--) {
        if (basis[i]) {
            if (k > (cnt >> 1)) {
                // set the ith bit
                if (!((x >> i) & 1)) {
                    x ^= basis[i];
                }
                k -= (cnt >> 1);
            }
            else {
                // unset the ith bit
                if ((x >> i) & 1) {
                    x ^= basis[i];
                }
            }
            cnt >>= 1;
        }
    }
    return x;
}
// numof occ of every xor=2^n-sz
// for l, r queries:
// quer[r].pushback({l, i}), traverse l to n

```

9 Number Theory

9.1 crt

```

#include <iostream>
#include <vector>
using namespace std;
// Chinese Remainder Theorem
int chineseRemainder(const vector<int> &num, const
    vector<int> &rem) {
    int prod = 1;
    for (int n : num) prod *= n;
    int result = 0;
    for (int i = 0; i < num.size(); ++i) {
        int pp = prod / num[i];
        result += rem[i] * modInverse(pp, num[i]) *
            pp;
    }
    return result % prod;
}
int main() {
    vector<int> num = {3, 4, 5}; // moduli
    vector<int> rem = {2, 3, 1}; // remainders
    cout << "x = " << chineseRemainder(num, rem) <<
        endl;
    return 0;
}

```

9.2 extendedEuclidean

```

typedef pair<int, int> pii;
#define x first
#define y second
pii extendedEuclid(int a, int b) { // returns x, y | ax
    + by = gcd(a,b)
    if (b == 0) return pii(1, 0);
    else {
        pii d = extendedEuclid(b, a % b);
        return pii(d.y, d.x - d.y * (a / b));
    }
}

```

9.3 FastEratosthenes

```

const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1);
    pr.reserve(int(LIM/log(LIM)*1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i)
            sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p))
                block[i-L] = 1;
        rep(i, 0, min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}

```

9.4 FloorSum

```

// 0(log a) sum^n floor(ax + b / c) = f
long long FloorSumAP(long long a, long long b, long
    long c, long long n) {
    if(!a) return (b / c) * (n + 1);
    if(a >= c or b >= c) return ((n * (n + 1)) / 2)
        * (a / c) + (n + 1) * (b / c) + FloorSumAP(a %
        c, b % c, c, n);
    long long m = (a * n + b) / c;
    return m * n - FloorSumAP(c, c - b - 1, a, m - 1);
}
// 0(log a) sum^n x * floor(ax + b / c) = g, sum^n
    floor(ax + b / c)^2 = h
struct dat {
    long long f, g, h;
    dat(long long f = 0, long long g = 0, long long h
        = 0) : f(f), g(g), h(h) {};
    long long mul(long long a, long long b) {
        return (a * b) % MOD;
    }
    dat query(long long a, long long b, long long c, long
        long n) {
        if(!a) return {mul(n + 1, b / c), mul(mul(mul(b /
        c, n), n + 1), inv2), mul(mul(n + 1, b / c), b
        / c)};
        long long f, g, h; dat nxt;
        if(a >= c or b >= c) {
            nxt = query(a % c, b % c, c, n);
            f = (nxt.f + mul(mul(mul(n, n + 1), inv2), a /
            c) + mul(n + 1, b / c)) % MOD;
            g = (nxt.g + mul(a / c, mul(mul(n, n + 1),
            mul(2 * n + 1, inv6))) + mul(mul(b / c,
            mul(n, n + 1), inv2)) % MOD;
            h = (nxt.h + 2 * mul(b / c, nxt.f) + 2 * mul(a
            / c, nxt.g) + mul(mul(a / c, a / c),
            mul(mul(n, n + 1), mul(2 * n + 1, inv6)))
            + mul(mul(b / c, b / c), n + 1) +
            mul(mul(a / c, b / c), mul(n, n + 1)) ) %
            MOD;
            return {f, g, h};
        }
        long long m = (a * n + b) / c;
        nxt = query(c, c - b - 1, a, m - 1);
        f = (mul(m, n) - nxt.f) % MOD;
        g = mul(mul(m, mul(n, n + 1)) - nxt.h - nxt.f,
            inv2);
        h = (mul(n, mul(m, m + 1)) - 2 * nxt.g - 2 * nxt.f
            - f) % MOD;
    }
}

```

```
return {f, g, h};}
```

9.5 millerRaben

```

typedef long long ll;
ll mulmod(ll a, ll b, ll c) {
    ll x = 0, y = a % c;
    while (b) {
        if (b & 1) x = (x + y) % c;
        y = (y << 1) % c;
        b >>= 1;
    }
    return x % c;
}
bool isPrime(ll n) {
    ll d = n - 1;
    int s = 0;
    while (d % 2 == 0) {
        s++; d >>= 1;
    }
    // It's guranteed that these values will work for
    // any number smaller than 3*10**18 (3 and 18
    // zeros)
    int a[9] = { 2, 3, 5, 7, 11, 13, 17, 19, 23 };
    for (int i = 0; i < 9; i++) {
        bool comp = binpow(a[i], d, n) != 1;
        if (comp) for (int j = 0; j < s; j++) {
            ll fp = binpow(a[i], (1LL << (ll)j)*d, n);
            if (fp == n - 1) {
                comp = false;
                break;
            }
        }
        if (comp) return false;
    }
    return true;
}

```

9.6 NumberTheory

```

// Description: Number theory utilities for computing
// GCD/LCM pair counts.
// Computes gp[i] = pairs with gcd=i, lp[i] = pairs
// with lcm=i.
// Time: $O(N \log N)$
const int MOD = 1e9 + 7;
struct NumberTheory {
    using ll = long long;
    vector<int> cnt;
    vector<ll> cm, cd, gp, lp;
    NumberTheory(int N) {
        cnt.resize(N + 1); cm.resize(N + 1);
        cd.resize(N + 1); gp.resize(N + 1); lp.resize(N + 1);
    }
    void compute(int n, const vector<int>& a) {
        for (int x : a) if (x <= n) cnt[x]++;
        for (int i = 1; i <= n; ++i)
            for (int j = i; j <= n; j += i)
                cm[i] += cnt[j], cd[j] += cnt[i];
        for (int i = n; i >= 1; --i) {
            gp[i] = (cm[i] * (cm[i] - 1)) / 2 % MOD;
            for (int j = 2 * i; j <= n; j += i)
                gp[i] = (gp[i] - gp[j] + MOD) % MOD;
        }
        for (int i = 1; i <= n; ++i)
            lp[i] = (cd[i] * (cd[i] - 1)) / 2 % MOD;
        for (int i = 1; i <= n; ++i)
            for (int j = 2 * i; j <= n; j += i)
                lp[j] = (lp[j] - lp[i] + MOD) % MOD;
    }
    void clear(int n) {
        for (int i = 1; i <= n; ++i)
            cnt[i] = cm[i] = cd[i] = gp[i] = lp[i] =
                0;
    }
}

```

9.7 PrimitiveRoot

```
// Author: Cp algorithm
```

```

// Description: Primitive root of a prime number.
// Time: O(sqrt(n) * log(n))
int generator(int p) {
    vector<int> fact;
    int phi = p - 1, n = phi;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0) n /= i;
        }
    if (n > 1) fact.push_back(n);
    for (int res = 2; res <= p; ++res) {
        bool ok = true;
        for (size_t i = 0; i < fact.size(); ++i)
            ok &= powmod(res, phi / fact[i], p) != 1;
        if (ok) return res;
    }
    return -1;
}

```

9.8 recursive divisor generate

```

vector<ll> SPF(N, 0ll);
vector<ll> primes;
void BuildSPF() {
    for (ll i = 2; i <= N; ++i) {
        if (SPF[i] == 0) {
            SPF[i] = i;
            primes.push_back(i);
        }
        for (ll j = 0; i * primes[j] <= N; ++j) {
            SPF[i * primes[j]] = primes[j];
            if (primes[j] == SPF[i]) break;
        }
    }
    bool is_prime(ll n) {
        if (n == 0) return false;
        return (SPF[n] == n);
    }
    void divisor_generate(ll idx, ll current_divisor,
        vector<pair<ll, ll>& factors,
        vector<ll>& result) {
        if (idx == factors.size()) {
            result.push_back(current_divisor); return;
        }
        auto [p, exp] = factors[idx];
        ll p_pow = 1;
        for (ll i = 0; i <= exp; ++i) {
            divisor_generate(idx + 1, p_pow * current_divisor,
                factors, result);
            p_pow *= p;
        }
    }
    vector<ll> get_divisors(ll n) {
        if (n == 1) return {1};
        vector<ll> divisors;
        vector<pair<ll, ll>> factors;
        while (n > 1) {
            ll p = SPF[n]; ll cnt = 0;
            while (n % p == 0) { n /= p; cnt++; }
            factors.push_back({p, cnt});
            divisor_generate(0, 1, factors, divisors);
            sort(divisors.begin(), divisors.end());
            return divisors;
        }
    }
}

```

9.9 Sieve

```

// call SIEVE() and PRIMES() in the int main()
const ll N = 1e6 + 5;
vector<bool> sieve(N, true);
vector<ll> primes;
void SIEVE() {
    sieve[1] = false;
    for (ll i = 2; i * i <= N; ++i) {
        if (!sieve[i]) continue;
        for (ll j = i * i; j <= N; j += i) {
            sieve[j] = false;
        }
    }
}

```

```
void PRIMES(){
    loop(1,2,N){
        if(sieve[i]){
            primes.pb(i);}}}

```

9.10 SquareFreeCount

```
//Description: Count square-free integers up to n using
//  Mobius inversion.
//Time:  $O(\sqrt{n})$ 
//Usage: ll cnt = squarefree_count(n, presum); //
//  presum[i] = prefix sum of mobius
ll squarefree_count(ll n, vector<ll>& presum) {
    ll ans = 0, r = 0;
    for (ll l = 1; l * l <= n; l = r + 1) {
        r = sqrt(n / (l * l));
        ans += (n / (l * l)) * (presum[r] - presum[l - 1]);
    }
    return ans;
}
```

9.11 Totient Values

```
vector<ll> phi(N+1,0ll);
//call phi 1 to n(N) in int main
void phi_1_to_n(int n) {
    for(int i=0;i<n;i++) phi[i]=i;
    for(int i=2;i<n;i++){
        if(phi[i]!=i) continue;
        for(int j=i;j<n;j+=i)phi[j]-=phi[j]/i;}}

```

10 String

10.1 KMP and Z Function

```
vector<ll> KMP(string&s){
    ll n=(ll)s.size();vector<ll> pi(n);
    for (int i=1;i<n;i++) {
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j])j++;
        pi[i]=j;
    }return pi;}
vector<ll> z_function(string&s){
    ll n=s.size();
    vector<ll> z(n);
    ll l=0,r=0;
    for(ll i=1;i<n;i++){
        if(i<r) z[i]=min(r-i,z[i-l]);
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
        if(i+z[i]>r){l=i;r=i+z[i];}
    }return z;}

```

10.2 Manacher

```
struct Manacher{
    vector<int> odd, even;
    ll num=0;
    string s;
    void build(string &given){
        s=given;
        num=s.size();
        odd.resize(num),even.resize(num);
        int left=0,right=-1;
        for(int i=0;i<num;i++){
            int k = i>right?
                1:min(odd[left+right-i],right+1-i);
            while(i+k<num && i-k>=0 &&
                s[i+k]==s[i-k])++k;
            odd[i]=k;
        }
    }
};

```

```
if(i+k-1>right)left=i-k+1,right=i+k-1;
left=0,right=-1;
for(int i=0;i<num;i++){
    int k=i>right?0:min(even[left+right-i+1],r,
        ight-i+1);
    while(i+k<num && i-k-1>=0 &&
        s[i+k]==s[i-k-1])++k;
    even[i]=k;
    if(i+k-1>right)left=i-k, right = i+k-1;}}
void longest_palindrome(){
    ll max_len=0,start=0;
    for(int i=0;i<num;i++){
        if(odd[i]*2-1>max_len){
            max_len=odd[i]*2-1;
            start=i+1-odd[i];}
        if(even[i]*2>max_len){
            max_len=even[i]*2;
            start=i-even[i];}}
    cout<<s.substr(start,max_len)<<endl;
};

```

10.3 MinRotation

```
//Description: Finds the lexicographically smallest
//  rotation of a string.
// Time:  $O(N)$ , Usage:
// rotate(v.begin(),v.begin()+minRotation(v),v.end());
int minRotation(string s) {
    int a=0, N=sz(s); s += s;
    rep(b,0,N) rep(k,0,N) {
        if (a+k == b || s[a+k] < s[b+k]) {b += max(0,
            k-1); break;}
        if (s[a+k] > s[b+k]) {a = b; break; }
    }
    return a;
}
```

10.4 PolyHash

```
#define MAXLEN 1000006
constexpr uint64_t mod=(1ULL<<61)-1;
const uint64_t seed=chrono::system_clock::now().time_s()
    ince epoch().count();
const uint64_t base=mt19937_64(seed)%(mod/3)+(mod/3);
uint64_t base_pow[MAXLEN];
int64_t modmul(uint64_t a,uint64_t b){
    uint64_t l1=(uint32_t)a,h1=a>>32,l2=(uint32_t)b,h2;
    l1=b>>32;
    uint64_t l=l1*l2,m=l1*h2+l2*h1,h=h1*h2;
    uint64_t ret=(l&mod)+(l>>61)+(h<<3)+(m>>29)+(m<<35);
    ret=(ret&mod)+(ret>>61);
    ret=(ret&mod)+(ret>>61);
    return ret-1;}
void init(){
    base_pow[0]=1;
    for(int i=1;i<MAXLEN;i++)
        base_pow[i]=modmul(base_pow[i-1],base);}
struct PolyHash{
    vector<int64_t> pref,suff;
    PolyHash(){}
    template<typename T>
    PolyHash(const vector<T>& ar){
        if(!base_pow[0]) init();
        int n=ar.size();
        assert(n<MAXLEN);
        pref.resize(n+3,0);
        suff.resize(n+3,0);
        for(int i=1;i<=n;i++){
            pref[i]=modmul(pref[i-1],base)+ar[i-1]+997;
            if(pref[i]>=mod) pref[i]-=mod;}
    }
};

```

```
// for(int i=n;i>=1;i--){
//
//     suff[i]=modmul(suff[i+1],base)+ar[i-1]+997;
//     if(suff[i]>=mod) suff[i]-=mod;
// }
PolyHash(string& str):PolyHash(vector<char>(str.begin(),
    str.end())){}
uint64_t get_hash(int l,int r){
    int64_t h=pref[r+1]-modmul(base_pow[r-l+1],pref[
        l]);
    return h<0?h+mod:h;}
uint64_t rev_hash(int l,int r){
    int64_t h=suff[l+1]-modmul(base_pow[r-l+1],suff[
        r+2]);
    return h<0?h+mod:h;}
};

```

10.5 Prefix Automation

```
// LCS of s1, s2 without vr as a substring
vector<int> prefix_function(string &s) {
    int n = (int)s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j]) j = pi[j-1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;}
int aut[N][26];
void compute_automaton(string s) {
    s += '#';
    int n = (int)s.size();
    vector<int> pi = prefix_function(s);
    for (int i = 0; i < n; i++) {
        for (int c = 0; c < 26; c++) {
            if (i > 0 && 'a' + c != s[i]) aut[i][c] =
                aut[pi[i-1]][c];
            else aut[i][c] = i + ('a' + c == s[i]);}}
string s1, s2, vr;
int dp[N][N][N];
int func(int i, int j, int k){
    if(k == vr.size()) return -200;
    if(i == s1.size() || j == s2.size()) return 0;
    int &ret = dp[i][j][k];
    if(ret != -1) return ret;
    ret = 0;
    if(s1[i] == s2[j]) ret = max(ret, 1 + func(i + 1,
        j + 1, aut[k][s1[i] - 'a']));
    ret = max({ret, func(i + 1, j, k), func(i, j + 1,
        k)});
    return ret;}
int mx;
void print(int i, int j, int k){
    if(mx == 0) return;
    if(s1[i] == s2[j]){
        if(func(i + 1, j + 1, aut[k][s1[i] - 'A']) ==
            mx - 1){
            cout << s1[i];
            mx--;
            print(i + 1, j + 1, aut[k][s1[i] - 'a']);}
    }
    if(func(i + 1, j, k) == mx) print(i + 1, j, k);
    else print(i, j + 1, k);}

```

10.6 SuffixArray

```

void inducedSort (const vector<int> &vec, int
    val_range, vector<int> &SA, const vector<int>
    &sl, const vector<int> &lms_idx) {
    vector<int> l(val_range, 0), r(val_range, 0);
    for (int c : vec) {
        ++r[c]; if (c + 1 < val_range) ++l[c + 1];
    }
    partial_sum(l.begin(), l.end(), l.begin());
    partial_sum(r.begin(), r.end(), r.begin());
    fill(SA.begin(), SA.end(), -1);
    for (int i = lms_idx.size() - 1; i >= 0; --i)
        SA[--r[vec[lms_idx[i]]]] = lms_idx[i];
    for (int i : SA) if (i > 0 and sl[i - 1])
        SA[l[vec[i - 1]]++] = i - 1;
    fill(r.begin(), r.end(), 0);
    for (int c : vec) ++r[c];
    partial_sum(r.begin(), r.end(), r.begin());
    for (int k = SA.size() - 1, i = SA[k]; k >= 0, i =
        SA[k]) {
        if (i and !sl[i - 1]) SA[--r[vec[i - 1]]] = i
            - 1;
    }
}

vector<int> suffixArray (const vector<int> &vec, int
    val_range) {
    const int n = vec.size();
    vector<int> sl(n), SA(n), lms_idx;
    for (int i = n - 2; i >= 0; --i) {
        sl[i] = vec[i] > vec[i + 1] or (vec[i] ==
            vec[i + 1] and sl[i + 1]);
        if (sl[i] and !sl[i + 1])
            lms_idx.emplace_back(i + 1);
    }
    reverse(lms_idx.begin(), lms_idx.end());
    inducedSort(vec, val_range, SA, sl, lms_idx);
    vector<int> new_lms_idx(lms_idx.size(),
        -1);
    for (int i = 0, k = 0; i < n; ++i) {
        if (SA[i] > 0 and !sl[SA[i]] and sl[SA[i] -
            1]) new_lms_idx[k++] = SA[i];
    }
    int cur = 0; SA[n - 1] = 0;
    for (int k = 1; k < new_lms_idx.size(); ++k) {
        int i = new_lms_idx[k - 1], j = new_lms_idx[k];
        if (vec[i] < vec[j]) {
            SA[j] = ++cur; continue;
        }
        bool flag = 0;
        for (int a = i + 1, b = j + 1; ; ++a, ++b) {
            if (vec[a] < vec[b]) {
                flag = 1; break;
            }
            if ((!sl[a] and sl[a - 1]) or (!sl[b] and
                sl[b - 1])) {
                flag = 1; break;
            }
            if (!sl[a] and sl[a - 1] and
                !sl[b] and sl[b - 1]) {
                flag = 1; break;
            }
        }
        SA[j] = flag ? ++cur : cur;
    }
    for (int i = 0; i < lms_idx.size(); ++i)
        lms_idx[i] = SA[lms_idx[i]];
    if (cur + 1 < lms_idx.size()) {
        auto lms_SA = suffixArray(lms_vec, cur + 1);
        for (int i = 0; i < lms_idx.size(); ++i)
            new_lms_idx[i] = lms_idx[lms_SA[i]];
    }
    inducedSort(vec, val_range, SA, sl, new_lms_idx);
    return SA;
}

vector<int> getSuffixArray (const string &s, const
    int LIM = 128) {
    vector<int> vec(s.size() + 1);
    copy(begin(s), end(s), begin(vec)); vec.back() =
        '$';
    auto ret = suffixArray(vec, LIM);
    ret.erase(ret.begin()); return ret;
}
// build RMQ on it to get LCP of any two suffix

```

```

vector<int> getLCPArray (const string &s, const
    vector<int> &SA) {
    int n = s.size(), k = 0;
    vector<int> lcp(n), rank(n);
    for (int i = 0; i < n; ++i) rank[SA[i]] = i;
    for (int i = 0; i < n; ++i, k ? --k : 0) {
        if (rank[i] == n - 1) {
            k = 0; continue;
        }
        int j = SA[rank[i] + 1];
        while (i + k < n and j + k < n and s[i + k] ==
            s[j + k]) ++k;
        lcp[rank[i]] = k;
    }
    lcp[n - 1] = 0; return lcp;
}
// s = s + "$", Append this after the string

```

10.7 SuffixAuto

```

struct SuffixAutomaton {
    struct State {
        int len, link;
        map<char, int> next;
        int first_pos;
        int cnt, cnt_paths;
    };
    vector<State> st;
    int last;
    SuffixAutomaton(string &s) {
        st.push_back({0, -1, {}, 0, 0, 0});
        last = 0;
        for (char c : s) {
            int cur = st.size();
            st.push_back({st[last].len + 1, 0, {},
                st[last].len + 1, 1, 0});
            int p = last;
            while (p != -1 && !st[p].next[c]) {
                st[p].next[c] = cur;
                p = st[p].link;
            }
            if (p == -1) {
                st[cur].link = 0;
            }
            else {
                int q = st[p].next[c];
                if (st[q].len == st[p].len + 1) {
                    st[cur].link = q;
                }
                else {
                    int clone = st.size();
                    st.push_back({st[p].len + 1,
                        st[q].link, st[q].next,
                        st[q].first_pos, 0, 0});
                    while (p != -1 && st[p].next[c] == q) {
                        st[p].next[c] = clone;
                        p = st[p].link;
                    }
                    st[q].link = st[cur].link = clone;
                }
            }
            last = cur;
        }
    }
    vector<int> count(st.size(), 0);
    for (int i = 0; i < st.size(); i++)
        count[st[i].len]++;
    for (int i = 1; i < st.size(); i++) count[i] +=
        count[i - 1];
    vector<int> order(st.size());
    for (int i = st.size() - 1; i >= 0; i--)
        order[--count[st[i].len]] = i;
    for (int i = st.size() - 1; i >= 0; i--)
        st[st[order[i]].link].cnt += st[order[i]].cnt;
    for (int i = st.size() - 1; i >= 0; i--) {
        int v = order[i];
        st[v].cnt_paths = 1;
        // For non-distinct substr, st[v].cnt_paths =
        // st[v].cnt;
        for (auto &p : st[v].next) {
            st[v].cnt_paths +=
                st[p.second].cnt_paths;
        }
    }
}

```

```

bool check_occur(string &t) {
    int cur = 0;
    for (char c : t) {
        if (!st[cur].next.count(c)) return 0;
        cur = st[cur].next[c];
    }
    return 1;
}

ll num_of_occur(string &t) {
    int cur = 0;
    for (char c : t) {
        if (!st[cur].next.count(c)) return 0;
        cur = st[cur].next[c];
    }
    return st[cur].cnt;
}

ll num_of_dif_substr() {
    ll sub = 0;
    for (int i = 1; i < st.size(); i++) {
        sub += st[i].len - st[st[i].link].len;
    }
    return sub;
}

ll tot_len_of_dif_substr() {
    ll total_length = 0;
    for (int i = 1; i < st.size(); i++) {
        ll shortest_substr_len = st[st[i].link].len +
            1;
        ll longest_substr_len = st[i].len;
        ll num_of_substr = longest_substr_len -
            shortest_substr_len + 1;
        total_length += num_of_substr *
            (longest_substr_len + shortest_substr_len)
            / 2;
    }
    return total_length;
}

string kth_substring(ll k) {
    ll tot_substr = 0;
    for (auto &p : st[0].next) {
        tot_substr += st[p.second].cnt_paths;
    }
    if (tot_substr < k) return "No such line.";
    string ans = "";
    int cur = 0;
    while (k > 0) {
        for (auto &p : st[cur].next) {
            int v = p.second;
            if (st[v].cnt_paths >= k) {
                ans.push_back(p.first);
                k -= st[v].cnt;
                cur = v;
                break;
            }
            else {
                k -= st[v].cnt_paths;
            }
        }
    }
    return ans;
}

string smallest_cyclic_shift(string &s) {
    string ss = s + s;
    SuffixAutomaton sa(ss);
    int n = s.length();
    string ans = "";
    int cnt = 0, cur = 0;
    while (cnt < n) {
        char cur_ch = 'z';
        for (auto &p : sa.st[cur].next) {
            if (p.second > 0) {
                cur_ch = p.first;
                cur = p.second;
                break;
            }
        }
        ans.push_back(cur_ch);
        cnt++;
    }
    return ans;
}

int first_occur(string &t) {
    int cur = 0;
    for (char c : t) {
        if (!st[cur].next.count(c)) return -1;
        cur = st[cur].next[c];
    }
    return st[cur].first_pos - t.length() + 1;
}

vector<int> all_occurrence_positions(string &t) {
    // ...
}

```

```

vector<int> positions;
int cur = 0;
for (char c : t) {
    if (!st[cur].next.count(c)) return positions;
    cur = st[cur].next[c];
}
vector<vector<int>> inv link(st.size());
for (int v = 1; v < st.size(); v++) {
    if (st[v].link != -1)
        inv link[st[v].link].push_back(v);
}
vector<bool> is clone(st.size(), true);
int org_str = 1;
is clone[0] = is clone[1] = 0;
while (!st[org_str].next.empty()) {
    for (auto &p : st[org_str].next) {
        if (st[p.second].len == st[org_str].len +
            1) {
            is clone[p.second] = 0;
            org_str = p.second;
        }
    }
}
function<void(int)> all occur = [&](int cur) {
    if (!is clone[cur]) {
        positions.push_back(st[cur].first_pos -
            t.length() + 1);
        for (int v : inv link[cur]) {
            all occur(v);
        }
    }
    all occur(cur);
    return positions;
}
string lon_com_substr(string &t) {
    int v = 0, l = 0, best = 0, best_pos = 0;
    for (int i = 0; i < t.size(); i++) {
        while (v && !st[v].next.count(t[i])) {
            v = st[v].link;
            l = st[v].len;
        }
        if (st[v].next.count(t[i])) {
            v = st[v].next[t[i]];
            l++;
        }
        if (l > best) {
            best = l;
            best_pos = i;
        }
    }
    return t.substr(best_pos - best + 1, best);
}
int lon_com_substr_multiple(vector<string> &str) {
    int ans = 0;
    vector<int> minmax(st.size(), str[0].size() + 1);
    for (int i = 1; i < str.size(); i++) {
        int cur = 0, len = 0;
        vector<int> mx(st.size(), 0);
        for (int j = 0; j < str[i].size(); j++) {
            while (cur && !st[cur].next.count(str[i][j])) {
                cur = st[cur].link;
                len = st[cur].len;
            }
            if (st[cur].next.count(str[i][j])) {
                cur = st[cur].next[str[i][j]];
                len++;
            }
            mx[cur] = max(mx[cur], len);
        }
        for (int i = 1; i < st.size(); i++) {
            minmax[i] = min(minmax[i], mx[i]);
        }
        for (int i = 1; i < st.size(); i++) {
            ans = max(ans, minmax[i]);
        }
    }
    return ans;
}
vector<ll> substr_by_len(int len) {
    vector<ll> res(len + 5, 0);
    vector<int> min len(st.size(), 0);
    //Also can find using bfs. CSES 1st submission
    for (int i = 1; i < st.size(); i++) {
        min len[i] = st[st[i].link].len + 1;
    }
    //Also can use seg tree lazy update. As there is
    //only one
    //query after all updates. So, this process works.
    for (int i = 1; i < st.size(); i++) {
        res[min len[i]]++;
        res[st[i].len + 1]--;
    }
}

```

```

for (int i = 1; i <= len; i++) {
    res[i] += res[i - 1];
}
return res;
}
void longest_repeating_substring(string &s) {
    int max_len = 0;
    int best_state = -1;
    for (int i = 1; i < st.size(); i++) {
        if (st[i].cnt > 1 && st[i].len > max_len) {
            max_len = st[i].len;
            best_state = i;
        } else if (st[i].cnt > 1 && st[i].len ==
            max_len) {
            int len = st[i].len;
            string current_substring =
                st[best_state].first_pos == -1 ? "" :
                s.substr(st[best_state].first_pos -
                    len, len);
            string candidate_substring =
                st[i].first_pos == -1 ? "" :
                s.substr(st[i].first_pos - len, len);
            if (candidate_substring <
                current_substring) {
                best_state = i;
            }
        }
    }
    if (best_state == -1) {
        cout << "-1";
        return;
    }
    int len = st[best_state].len;
    int pos = st[best_state].first_pos;
    cout << s.substr(pos - len, len) << " ";
}
void display() {
    cout << "State : Link, length, Transitions" <<
        endl;
    for (int i = 0; i < st.size(); i++) {
        cout << i << " : " << st[i].link << ", " <<
            st[i].len << " ";
        for (auto &p : st[i].next) {
            cout << p.first << "->" << p.second << "
                ";
        }
        cout << endl;
    }
}

```

10.8 trieString

USE CASE:
Fast string insertion, search, and prefix counting.
O(L) complexity for all operations (L = string length).
QUICK GUIDE:
insert("apple"): Adds "apple". Increments .pass for
"a", "p", "p", "l", "e" and .end for "e".
find("apple"): Returns true ONLY if "apple" was
explicitly inserted.
prefixmatchcount("app"): Returns count of ALL strings
starting with "app" (e.g., apple, apply).
erase("apple"): Removes ONE instance. Cleanly deletes
unused nodes.
CAUTION:
id(char c) assumes 'a'-'z'. Inputting 'A' or '1' will
SEGFAULT.
Memory: Each node uses 26 * 8 bytes (pointers). Space
grows fast.

```

struct Trie {
    static const ll SIG = 26;
    struct Node {
        Node *nxt[SIG];
        ll pass; // how many strings pass through this node
        ll end; // how many strings end at this node
        Node() : pass(0), end(0) { memset(nxt, 0, sizeof(nxt)); }
    };
    Node *root;
    Trie() { root = new Node(); }
    ~Trie() { clear(root); }
}

```

```

static inline ll id(char c) { return c - 'a'; }
//make sure letter is lowercase or witness segfault
void insert(const string &s) {
    Node* cur = root;
    cur->pass++;
    for (char c : s) {
        ll k = id(c);
        if (!cur->nxt[k]) cur->nxt[k] = new Node();
        cur = cur->nxt[k];
        cur->pass++;
    }
    cur->end++;
}
bool find(const string &s) const {
    const Node *cur = root;
    for (char c : s) {
        ll k = id(c);
        if (!cur->nxt[k]) return false;
        cur = cur->nxt[k];
    }
    return (cur->end) > 0;
}
ll prefixmatchcount(const string &s) const {
    const Node *cur = root;
    for (char c : s) {
        ll k = id(c);
        if (!cur->nxt[k]) return 0;
        cur = cur->nxt[k];
    }
    return cur->pass;
}
// erase ONE occurrence of s; returns true if something
// was erased
bool erase(const string &s) {
    if (!find(s)) return false;
    Node* cur = root;
    (cur->pass)--;
    vector<pair<Node*, ll>> path; // (parent,
        edge index)
    path.reserve(s.size());
    for (char c : s) {
        ll k = id(c);
        path.push_back({cur, k});
        cur = cur->nxt[k];
        cur->pass--;
    }
    cur->end--;
    // cleanup nodes from bottom if their pass becomes
    // 0
    for (ll i = (ll)path.size() - 1; i >= 0; i--) {
        Node* parent = path[i].first;
        ll k = path[i].second;
        Node* child = parent->nxt[k];
        if (child->pass == 0) {
            clear(child);
            parent->nxt[k] = nullptr;
        } else {
            break; // higher nodes still needed
        }
    }
    return true;
}
private:
void clear(Node *cur) {
    if (!cur) return;
    for (ll i = 0; i < SIG; i++) clear(cur->nxt[i]);
    delete cur;
}

```

11 zMiscellaneous

11.1 bitset_operations

```

bitset<8> b1(string("10101100"));
cout << "b1 = " << b1 << endl;
cout << (b1 & b2) << (~b1) << (b1 | b2) << (b1 ^ b2); // Output:
    00101100
b3.set(); // Set all bits
b3.reset(); // Reset all bits
b3.flip(); // Flip all bits
b3.set(2); // Set a specific bit (index 2)

```

```
b3.reset(2); // Reset a specific bit (index 2)
b3.flip(0); // Flip a specific bit (index 0)
// Count number of set bits : b1.count()
// Get value of a specific bit (index 3) : b1[3]
// Convert bitset to unsigned long : b1.to_ulong()
// Convert bitset to unsigned long long :
    ~ b1.to_ullong()
// Convert bitset to string : b1.to_string()
```

11.2 builtinOperations

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
//popcnt(x) (Population Count): Counts total '1' bits.
#define popcnt(x) __builtin_popcountll(x)
//ctz(x) (Count Trailing Zeros): Counts '0' from the
    ~ right up to the
//first '1'. 0-indexed position of the lowest set bit.
//Example: ctz(1100) = 2. (Undefined if x = 0).
#define ctz(x) __builtin_ctzll(x)
//clz(x) (Count Leading Zeros): Counts '0' from the
    ~ left down to the
//first '1'. (Undefined if x = 0). Find highest bit
    ~ position (0-indexed) using '63 - clz(x)'.
#define clz(x) __builtin_clzll(x)
//ffs(x) (Find First Set): Finds the position of the
    ~ first '1' bit from the
//right using 1-indexing.
//Example: ffs(1100) = 3. (Returns 0 if x = 0).
#define ffs(x) __builtin_ffsll(x)
//parity(x): Returns 1 if total '1' bits is odd; 0 if
    ~ even.
#define parity(x) __builtin_parityll(x)
```

11.3 CoordinateCompression

```
struct Compression{
    vector<ll> vals;
    void build(const vector<ll>& a){
        vals=a;
        sort(vals.begin(),vals.end());
        vals.erase(unique(vals.begin(),vals.end()),vals)
            ~ s.end());}
    int get(ll x){
        return lower_bound(vals.begin(),vals.end(),x)-
            ~ vals.begin();}
    ll rev(int i){
        return vals[i];}
    vector<int> compress_array(const vector<ll>& a){
        vector<int> res(a.size());
        for(int i=0;i<a.size();i++)
            res[i]=get(a[i]);
        return res;}
    pair<int,int> get_range(ll l,ll r){
        int L=lower_bound(vals.begin(),vals.end(),l)-v
            ~ als.begin();
        int R=upper_bound(vals.begin(),vals.end(),r)-v
            ~ als.begin()-1;
        if(L>R) return {-1,-1};
        return {L,R};};
```

11.4 CUSTOM HASH

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);}
```

```
size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch()
            ~ ch().count();
    return splitmix64(x + FIXED_RANDOM);
} //pair hash
template<typename T, typename U>
size_t operator()(const pair<T, U>& x) const {
    static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch()
            ~ ch().count();
    uint64_t h1 = splitmix64((uint64_t)x.first +
        ~ FIXED_RANDOM);
    uint64_t h2 = splitmix64((uint64_t)x.second +
        ~ FIXED_RANDOM);
    return h1 ^ (h2 >> 1);};
```

11.5 Custom_Comparator_for_pq

```
auto cmp = [](const pair<int, int>& a, const pair<int,
    ~ int>& b) {
    return a.first > b.first;
}; // Min-heap based on 'first'
priority_queue<pair<int, int>, vector<pair<int, int>>,
    ~ decltype(cmp)> pq(cmp);
```

11.6 fast_subset_sum

```
// Description: Bounded-multiplicity subset sum DP.
    ~ Returns bit[s]=1 if sum s is achievable.
// Time:  $\mathcal{O}(n \cdot \text{cdot} | \text{ext}\{\text{distinct values}\}|)$ 
// Usage: auto bit = fast_subset_sum(mp, n); // mp:
    ~ value->freq, n: max sum
vector<int> fast_subset_sum(const map<int,int>& mp,
    ~ int n){
    vector<pair<int,int>> v;
    for(auto &pr : mp) v.push_back({pr.first,
        ~ pr.second});
    sort(v.rbegin(), v.rend());;
    int m = (int)v.size();
    // dp[i][j] : using items from i..m-1 can we
        ~ achieve sum j
    vector<vector<bool>> dp(m + 1, vector<bool>(n + 1,
        ~ 0));
    vector<int> bit(n + 1, 0);
    dp[m][0] = 1;
    for(int i = m - 1; i >= 0; --i){
        auto [val, freq] = v[i];
        vector<int> lst(val, -1);
        for(int j = 0; j <= n; ++j) {
            if(dp[i + 1][j]) lst[j % val] = j;
            if(lst[j % val] == -1 || ((j - lst[j % val]) /
                ~ val) > freq){
                dp[i][j] = 0;
            } else {
                dp[i][j] = 1;
                bit[j] = 1;}}
        return bit;}
// Log Trick
// Given a[i], freq[i]
bitset<MAXX> dp;
dp[0] = 1;
for (int i = 1; i <= n; i++) {
    for (int x = 0; (1<<x) <= freq[i]; x++) {
        dp |= (dp << (a[i]*(1<<x)));
        m[i] -= (1<<x);}
    dp |= (dp << (a[i]*freq[i]));}
// now, dp[k] equals the k-th bit of f(n)
```

11.7 int128

```
using int128 = __int128_t; using uint128 = __uint128_t;
std::ostream &operator<< (std::ostream &os, __int128 n){
    if (n == 0) return os << "0";
    if (n < 0){os << "-"; n = -n;}
    std::string s;
    while (n > 0){s += (char)('0' + (n % 10)); n /= 10;}
    std::reverse(s.begin(), s.end());
    return os << s;
} std::istream &operator>> (std::istream &is, __int128 &n){
    std::string s; is >> s; n = 0;
    int i = 0; bool negative = false;
    if (s[0] == '-') {negative = true; i = 1;}
    for (; i < s.size(); i++){n = n * 10 + (s[i] - '0');}
    if (negative)n = -n;
    return is;
} namespace Int128Utils{int128 abs128(int128 n) { return
    ~ n < 0 ? -n : n; }}
```

11.8 pbds

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T>
using ordered_set = tree<T, null_type, less<T>,
    ~ rb_tree_tag, tree_order_statistics_node_update>;
template<typename dataType1, typename dataType2>
using ordered_map = tree<dataType1, dataType2,
    ~ less<dataType1>, rb_tree_tag,
    ~ tree_order_statistics_node_update>;
// For ordered set of pairs
ordered_set<pair<int, int>> osp;
// For ordered set with custom comparator
struct custom_compare {
    bool operator() (const pair<int, int>& a, const
        ~ pair<int, int>& b) const {
        if (a.first != b.first) return a.first <
            ~ b.first;
        return a.second < b.second;};
tree<pair<int, int>, null_type, custom_compare,
    ~ rb_tree_tag, tree_order_statistics_node_update>
    ~ custom ordered set;
os.insert(5); // Insert element
os.erase(5); // Erase element
os.find(2)
// 1. find by order(k)
// returns iterator to k-th element (0-based)
*os.find by order(0)
// 2. order of key(x)
// returns number of elements STRICTLY LESS than x
os.order_of_key(2)
s.erase(5.lower_bound(val), s.upper_bound(val));
gp_hash_table<int, int> fast_map;
gp_hash_table<int, null_type> fast_set;
```

11.9 submaskiterate

```
// Description: Iterate through all submasks of each
    ~ mask from 0 to  $2^n - 1$ .
// For each mask m, iterates through all s where s is a
    ~ subset of m (s AND m = s).
// Time:  $\mathcal{O}(3^n)$ 
for (int m=0; m<(1<<n); ++m)
    for (int s=m; s; s=(s-1)&m)
        ... s and m ...
```